

DISCRETE MATHEMATICS FOR COMPUTER SCIENCE



Mark A. Fitch
University of Alaska Anchorage

Discrete Mathematics for Computer Science (Fitch)

This text is disseminated via the Open Education Resource (OER) LibreTexts Project (<https://LibreTexts.org>) and like the thousands of other texts available within this powerful platform, it is freely available for reading, printing, and "consuming."

The LibreTexts mission is to bring together students, faculty, and scholars in a collaborative effort to provide an accessible, and comprehensive platform that empowers our community to develop, curate, adapt, and adopt openly licensed resources and technologies; through these efforts we can reduce the financial burden born from traditional educational resource costs, ensuring education is more accessible for students and communities worldwide.

Most, but not all, pages in the library have licenses that may allow individuals to make changes, save, and print this book. Carefully consult the applicable license(s) before pursuing such effects. Instructors can adopt existing LibreTexts texts or Remix them to quickly build course-specific resources to meet the needs of their students. Unlike traditional textbooks, LibreTexts' web based origins allow powerful integration of advanced features and new technologies to support learning.



LibreTexts is the adaptable, user-friendly non-profit open education resource platform that educators trust for creating, customizing, and sharing accessible, interactive textbooks, adaptive homework, and ancillary materials. We collaborate with individuals and organizations to champion open education initiatives, support institutional publishing programs, drive curriculum development projects, and more.

The LibreTexts libraries are Powered by [NICE CXone Expert](#) and was supported by the Department of Education Open Textbook Pilot Project, the California Education Learning Lab, the UC Davis Office of the Provost, the UC Davis Library, the California State University Affordable Learning Solutions Program, and Merlot. This material is based upon work supported by the National Science Foundation under Grant No. 1246120, 1525057, and 1413739.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation nor the US Department of Education.

Have questions or comments? For information about adoptions or adaptations contact info@LibreTexts.org or visit our main website at <https://LibreTexts.org>.

This text was compiled on 06/09/2025

TABLE OF CONTENTS

Licensing

1: Basics

- 1.1: Terminology
- 1.2: Sets
- 1.3: Set Operations
- 1.4: Sets in Relational Database
- 1.5: Combinatorics- First Counts
- 1.6: Sorting- Part 0

2: Logic

- 2.1: Formal Logic
- 2.2: Circuits
- 2.3: Predicate Logic
- 2.4: Mathematical Proof

3: Functions

- 3.1: Functions
- 3.2: Boolean Algebra
- 3.3: Modulo Arithmetic
- 3.4: Combinatorics- Second Counts
- 3.5: Proof by Contradiction

4: Relations

- 4.1: Example- Error Correcting Codes
- 4.2: Mathematical Relations
- 4.3: Equivalence Relations
- 4.4: Partially Ordered Sets
- 4.5: Combinatorics- Inclusion/Exclusion

5: Graph Theory

- 5.1: Discovering Graphs
- 5.2: Properties of Graphs
- 5.3: Graph Isomorphism
- 5.4: Paths
- 5.5: Cycles
- 5.6: Trees
- 5.7: Linear Recurrence Relations

6: Appendix

- 6.1: A- University Database Example
- 6.2: B- Glossary

[Index](#)

[Glossary](#)

[Detailed Licensing](#)

Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

CHAPTER OVERVIEW

1: Basics

- [1.1: Terminology](#)
- [1.2: Sets](#)
- [1.3: Set Operations](#)
- [1.4: Sets in Relational Database](#)
- [1.5: Combinatorics- First Counts](#)
- [1.6: Sorting- Part 0](#)

This page titled [1: Basics](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.1: Terminology

One of the concerns of mathematics is demonstrating the truth of statements. These demonstrations are called proofs and this topic will be studied later. For the moment note that in any finite logic system (this is what humans use), there must be the following categories of terms and statements.

1. Undefined terms
2. Defined terms
3. Axioms
4. Theorems (lemmas, corollaries)

There must be undefined terms. If not then each term would be defined in terms of a previous one which would iterate infinitely. If the list is not infinite then some term is defined by one earlier in the list leading to a circular definition. This is just listing synonyms.

Defined terms are defined using undefined terms and previously defined terms. Axioms are unproven statements that explain how undefined and defined objects work.

These statements must be unproven for the same reason undefined terms must exist.

Finally, new statements can be proved using undefined terms, defined terms, axioms, and previously proven theorems. Each section will begin with a list of undefined and defined terms. These must be memorized in order to understand statements made and solve problems using them.

This page titled [1.1: Terminology](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.2: Sets

Understanding Sets

The most basic data structure is a set. Sets are defined by what is included or what is excluded. To understand what else this implies answer the following questions.

Checkpoint 1.2.1 For each of the following determine whether order matters or not.

1. Members of congress (U.S. federal)
2. Finishers of the Tour de France
3. Students enrolled at UAA
4. People in line at a coffee kiosk

Checkpoint 1.2.2 Given that sets indicate what is included and excluded what, if anything, is extraneous in the following sets?

1. { green, gold, gold, green } (context: set of colors)
2. {1,2,3,4,5,...} {1,2,3,4,5,...} (context: set of integers)
3. {0,1,1×1,1×0} {0,1,1×1,1×0} (context: set of integers)
4. Students enrolled in MATH A261 (context: set of students)

Checkpoint 1.2.3 Determine whether or not you are in the following sets.

1. UAA students
2. Computer Science majors
3. Millionaires
4. Musicians

In mathematics concepts must be "well defined." This means there are no ambiguities. For sets this means that it must be possible to determine whether every object is in a set or not. Note this does not mean that any person necessarily knows the answer, rather that some means exists. For example every number is prime or it is not. However, no person knows or can list all the primes.

Checkpoint 1.2.4 Are all of the sets above (Checkpoint 1.2.3) well defined?

Terminology

The terms **collection**, **set**, **member**, and **element** are undefined. **Collection** and **set** are synonyms. An element is an object that is included in a set. **Element** and **member** are synonyms. Because specific sets are defined by membership, sets do not have a concept of order of their elements.

The following are mathematical notations for defining sets.

Set	Element
$T = \{true, false\}$	$true \in T$
$E = \{n n = 2k, k \notin \mathbb{Z}\}$	$3 \notin E$
S is the set of students enrolled in MATH A261 in the fall 2019 semester	Guido is not in S

Table 1.2.5. Set Notations

Definition 1.2.6: Subset

A set A is a subset of a set B , denoted $A \subset B$, if and only if $a \in A$ implies $a \in B$.

For example $E \subset \mathbb{Z}$. Note when needed $X \subseteq Y$ means X is a subset but might be equal to Y whereas $X \subset Y$ means X is a subset and Y contains at least one element not in X . However, we rarely need to make this distinction in this class, so we will use $\subset$ to mean $\subseteq$.

Often we need to talk about a set that has no elements. This is called the **empty set** and is written as $\emptyset$.

Practice

Checkpoint 1.2.7 Determine if the first set is a subset of the second set.

1. even numbers, all integers
2. vowels, letters
3. colors, pastels
4. chairs, people

Checkpoint 1.2.8 Determine if the object is an element of the given set.

1. apple, fruit
2. spider, insects
3. bird shot (shotguns), things that float
4. bumble bees (living, healthy), things that can fly

This page titled [1.2: Sets](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.3: Set Operations

Terminology

Just as there are arithmetic operations on numbers, there are set operations on sets.

Definition 1.3.1: Union

The set C is the **union** of the sets A and B , denoted $A \cup B$, if and only if $C = \{x | x \in A \text{ or } x \in B\}$.

Example 1.3.2: Union

Consider $A = \{a, l, k, s\}$ and $B = \{a, c, l, k\}$. $A \cup B = \{a, c, l, k, s\}$ because $a \in A$, $c \in B$, $l \in A$, $k \in A$, and $s \in B$. Note it does not matter if we say because $a \in A$ or because $a \in B$. This is further discussed in [Section 2.1](#)

Checkpoint 1.3.3 Compare/Contrast.

Union can be thought of as taking the elements of the first set and adding any elements of the second set that are different. Our question is how does **union** differ from **addition**?

► Hint

Add texts here. Do not delete this text first. Try unions of $A = \{1, 2, 3\}$ with the following sets $B_1 = \{4, 5, 7\}$, $B_2 = \{3, 4, 5\}$, and $B_3 = \{2, 3\}$.

Definition 1.3.4: Intersection

The set C is the **intersection** of the sets A and B , denoted $A \cap B$, if and only if $C = \{x | x \in A \text{ and } x \in B\}$.

Watch the video in **Figure 1.3.5** demonstrating reading the set intersection definition twice. The first time think about what set intersection is. The second time think about how it illustrates reading and understanding a definition.



Figure 1.3.5: Understanding the set intersection definition

Definition 1.3.6: Difference

The set C is the **difference** between the sets A and B , denoted $A - B$, if and only if $C = \{x | x \in A \text{ but } x \notin B\}$.

Checkpoint 1.3.7 Compare/Contrast.

Just as we considered how union is like addition we can compare and contrast **set difference** with **subtraction**.

► Hint

Add texts here. Do not delete this text first. Try the set difference of $A = \{1, 2, 3\}$ with the following sets $B_3 = \{2, 3\}$, $B_2 = \{3, 4, 5\}$, and $B_1 = \{4, 5, 7\}$.

Checkpoint 1.3.8 Algorithmic complexity.

A programmer must consider how to implement an operation like set difference.

(a) Find $A - B$ for $A = \{1, 3, 5, 7, 9\}$ and $B = \{1, 3, 9\}$

(b) Set difference must remove from A every element in B . Consider the following psuedo-code.

```
for x in A do
  for y in B do
    if y==x then
      remove(A,x)
      exit inner for loop
    end if
  end for
end for
```

How many times will the comparison (`if x==y` line) be run for these sets A and B .

(c) If the psuedo-code algorithm were reversed so the outside loop were for B and the inside loop for A would it change the number of comparisons?

(d) Can you think of any optimizations of this algorithm?

In many circumstance there is a **universal set** that is part of the context for a problem. It may or may not be explicitly stated. For example the set of all integers may be the universal set for discussion of even and odd numbers.

Definition 1.3.9: Complement

The set C is the **complement** of the set A with respect to the universal set U , denoted $\neg A$ or $\overline{A}$, if and only if $C = U - A$.

Definition 1.3.10: Power Set.

The set P is the **power set** of the set A , denoted $P(A)$ or 2^A , if and only if P is the set of all subsets of A .

For example if $A = \{0, 1\}$ then $P(A) = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$.

Checkpoint 1.3.11. Developing Power Sets. Power sets are sets. This means they are unordered. However, to generate them it helps us list all subsets if we impose some convenient order. The following suggest parts of a heuristic for listing elements of power sets. We consider the power set of $S = \{a, b, c\}$.

(a) There are subsets of S of size 0,1,2, and 3. What is the only subset of size 0? What is the only subset of size 3?

(b) List all subsets of size 1.

(c) List all subsets of size 2.

(d) There are an equal number of subsets of size 1 and size 2 for S . Find a logical correspondence between the subsets of size 1 and size 2 (pair each subset of size 1 with a subset of size 2).

(e) In general for a set of size n how could one generate all subsets? Note the steps above only suggests two parts of an algorithm.

 Definition 1.3.12: Cartesian Product.

The set C is the Cartesian product of the sets A and B , denoted $A \times B$, if and only if $C = \{(a, b) | a \in A \text{ and } b \in B\}$.

For example if $A = \{0, 1\}$ and $B = \{a, b\}$ then $A \times B = \{(0, a), (0, b), (1, a), (1, b)\}$.

Note for a set A and positive integer $n \geq 1$ we use the notation A^n to mean n copies of A in a Cartesian product. For example, $A^3 = A \times A \times A$.

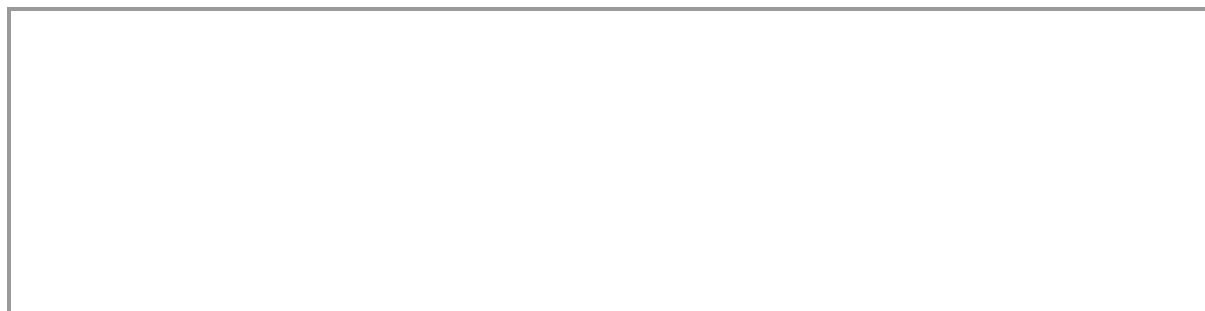
 Definition 1.3.13: Size of.

We define the size of a finite set A , denote $|A|$ to be the number of elements it contains.

For example if $A = \{a, b, c\}$ then $|A| = 3$. There are ways to define the size of infinite sets, but those must wait for later classes.

Checkpoint 1.3.14.

WeBWork: <http://aholiab.uaa.alaska.edu/webwor...VU1FTlQoKTs%3D>



1.3.2 Practice

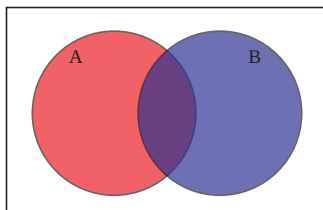


Figure 1.3.15: Typical Venn Diagram

Checkpoint 1.3.16 Identify the following in the diagram above.

1. $A \cup B$
2. $A \cap B$
3. $A - B$
4. $B - A$
5. $\neg A$
6. $\neg B$

Checkpoint 1.3.17 Calculate the following for $A = \{0, 1, 2\}$ and $B = \{0, 1\}$.

1. $P(A)$
2. $A \times B$
3. $B \times A$
4. $A - B$
5. $B - A$

Checkpoint 11.3.18 Perform the set operations indicated on the sets below. The universal set is all integers.

- $A = \{n | n = 2k \text{ for some integer } k\}$

- $B = \{n | n = 5k \text{ for some integer } k\}$
 - $C = \{n | n = 6k \text{ for some integer } k\}$
 - $D = \{n | n = 10k \text{ for some integer } k\}$
1. $A \cup B$
 2. $A \cap B$
 3. $C \cap D$
 4. $A - B$
 5. $B - A$
 6. $\neg A$
 7. $\neg B$

This page titled [1.3: Set Operations](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.4: Sets in Relational Database

Databases consist of sets of data. Tables are sets of structured data. Each "row" contains entries for one or more fields; each field is a specific type of data such as a string, integer, or date. One of the uses of databases is to connect tables based on shared fields. For an example see [Appendix A](#).

The most common method for accessing databases, especially large, enterprise databases, is Structured Query Language (SQL pronounced *sequel*). Below is a very simple illustration of the format of SQL query statements.

Table 1.4.1 SQL query example

SELECT [list of fields]	each field from each table desired
FROM [1st table]	
(INNER/LEFT/RIGHT/OUTER) JOIN [2nd table]	how to match rows of tables
WHERE <condition>	limits data used

A specific example that shows the midterm grade and placement test score of all students on the roll is below.

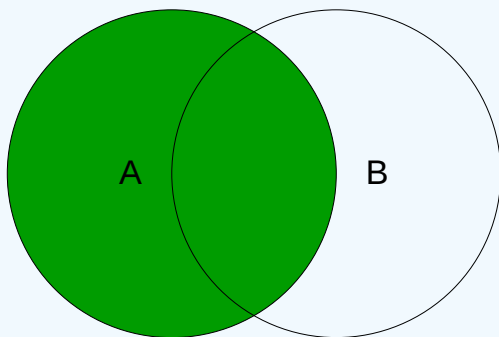
```
SELECT Midterm.Section, Compare.[Student ID],
       Compare.[Placement Score 1], Compare.[Placement Score 2],
       Midterm.Score, Midterm.Letter
FROM Compare
INNER JOIN Midterm ON Compare.[Student ID] = Midterm.ID;
```

Note that the JOIN statements determine what set operation to perform on the tables. For example a left join takes all rows from the 1st table regardless of whether there is a matching row in the second table. An inner join requires that there be a matching row in both tables. An outer join uses all rows from both tables regardless of whether there is a match.

✓ Example 1.4.2: SQL Left Join in Set Notation.

```
SELECT [fields]
FROM Table A LEFT JOIN Table B
ON A.Key=B.Key
```

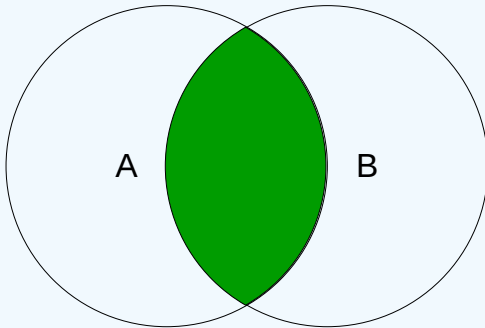
Set Notation: A



✓ Example 1.4.3: SQL Inner Join in Set Notation.

```
SELECT [fields]
FROM Table A INNER JOIN Table B
ON A.Key=B.Key
```

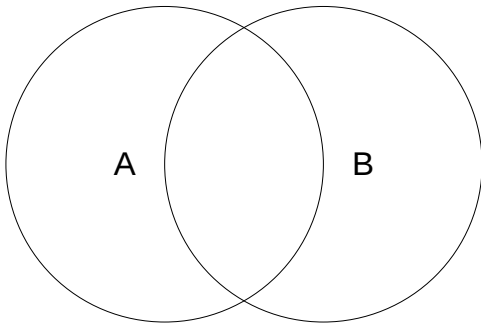
Set Notation: $A \cap B$



Checkpoint 1.4.4

```
SELECT [fields]
FROM Table A RIGHT JOIN Table B
ON A.Key=B.Key
```

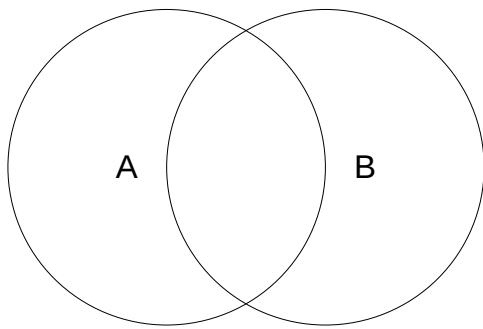
Set Notation: _____



Checkpoint 1.4.5

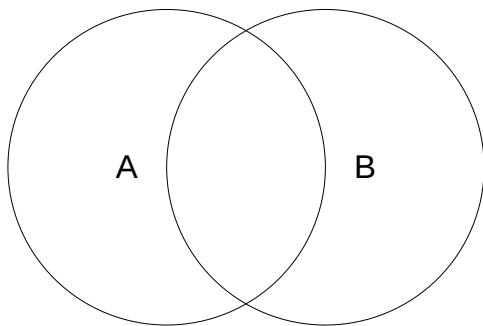
```
SELECT [fields]
FROM Table A LEFT JOIN Table B
ON A.Key=B.Key
WHERE B.Key IS NULL
```

Set Notation: _____

**Checkpoint 1.4.6**

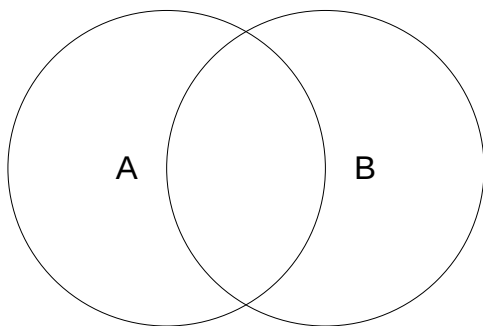
```
SELECT [fields]
FROM Table A RIGHT JOIN Table B
ON A.Key=B.Key
WHERE A.Key IS NULL
```

Set Notation: _____

**Checkpoint 1.4.7**

```
SELECT [fields]
FROM Table A FULL OUTER JOIN Table B
ON A.Key=B.Key
```

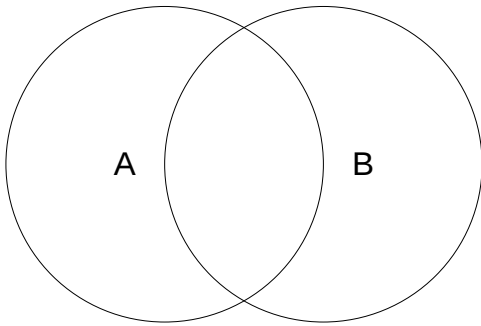
Set Notation: _____

**Checkpoint 1.4.8**

```
SELECT [fields]
FROM Table A FULL OUTER JOIN Table B
```

```
ON A.Key=B.Key  
WHERE A.Key IS NULL or B.Key IS NULL
```

Set Notation: _____



This page titled [1.4: Sets in Relational Database](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.5: Combinatorics- First Counts

Combinatorics is the mathematics of counting.

Terminology

Alphabet is an undefined term referring to a set of objects typically thought of as symbols such as the alphabets used in western European languages, Cyrillic, digits of numbers, or emojis.

Definition 1.5.1: String/Word.

A **string** or **word** is an ordered set of characters from an alphabet.

A few words from the alphabet $\{a, b, c\}$ are the following: c, abc, bc, cab, ccb, aaa. A few words from the alphabet $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ are 301, 112, 5.

Definition 1.5.2: Permutation.

A **permutation** of a string is any arrangement of the characters in that string including the same order.

A few of the permutations of the string “abc” are: abc, bac, cab.

Practice

Checkpoint 1.5.3 Generate a list of all possible strings of length two from the alphabet $\{0, 1\}$.

Checkpoint 1.5.4 How many strings of length three are possible from the alphabet $\{0, 1\}$? Do not use a formula.

Checkpoint 1.5.5 How many strings of length three are possible from the alphabet $\{0, 1, 2\}$? Do not use a formula.

Checkpoint 1.5.6 Write all permutations of the string “mat”.

Checkpoint 1.5.7 How many permutations are there of the string “eat”.

Checkpoint 1.5.8 Write all permutations of the string “math”.

Checkpoint 1.5.9 How many permutations are there of the string “heat”.

Checkpoint 1.5.10 How many permutations are there of the string “great”.

This page titled [1.5: Combinatorics- First Counts](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

1.6: Sorting- Part 0

If an alphabet has an order, then the strings can be ordered. This is called lexicographic ordering. For words from a language, we often call this alphabetical ordering. Because the order on the strings is determined by the order on the alphabet, we call the ordering induced.

Practice

Checkpoint 1.6.1 Write all the permutations of the string “mat”. List them in lexicographic order.

Checkpoint 1.6.2 The lexicographic order is just one of the orders. How many possible orders of the permutations of the string “mat” exist?

Checkpoint 1.6.3 How many possible orders of the permutations of the string “math” exist?

Checkpoint 1.6.4 As the length of the string increases, how much does the number of possible orders increase?

This page titled [1.6: Sorting- Part 0](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

2: Logic

[2.1: Formal Logic](#)

[2.2: Circuits](#)

[2.3: Predicate Logic](#)

[2.4: Mathematical Proof](#)

This page titled [2: Logic](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

2.1: Formal Logic

Terminology

The concept of **true** and **false** are undefined terms.

Definition 2.1.1: Statement

An expression is a **statement** if and only if it is either true or false.

The following are statements. "The sky is blue." "George Washington was the first president of the United States." The following are not statements. "Hello." "Study hard." "Is that true?" For convenience letters, such as A and B, are used to refer to statements. 0 represents false and 1 represents true

Statements can be modified and combined. **Figure 2.1.2** gives the definitions for some of these logic operations. A **truth table** is a notation for listing all possible inputs to a statement and the resulting truth or lack thereof. Below are truth tables defining the basic logic operations.

And		
A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

Or		
A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

Not	
A	not A
0	1

If		
A	B	If A, then B
0	0	1
0	1	1
1	0	0
1	1	1

Figure 2.1.2 Definition of Logic Operations

For convenience the following symbols are used for these operations. As with arithmetic operations, there is an order of operations. **Figure 2.1.3** lists the operators in first to last order.

highest not $\neg$
and $\wedge$
or $\vee$
lowest if $\rightarrow$

Figure 2.1.3 Logic Order of Operations

Practice

Checkpoint 2.1.4

Complete the following truth tables.

AA	BB	$A \rightarrow B$
0	0	
0	1	
1	0	
1	1	

AA	BB	$B \rightarrow A$

0	0	
0	1	
1	0	
1	1	

AA	BB	$\neg A \neg A$	$(\neg A) \rightarrow B(\neg A) \rightarrow B$
0	0		
0	1		
1	0		
1	1		

AA	BB	$\neg B \neg B$	$(\neg B) \rightarrow A(\neg B) \rightarrow A$
0	0		
0	1		
1	0		
1	1		

AA	BB	$\neg A \neg A$	$(\neg A) \vee B(\neg A) \vee B$
0	0		
0	1		
1	0		
1	1		

AA	BB	$\neg A \neg A$	$\neg B \neg B$	$(\neg B) \rightarrow (\neg A)(\neg B) \rightarrow (\neg A)$
0	0			
0	1			
1	0			
1	1			

Checkpoint 2.1.5

Complete the following truth table.

AA	BB	CC	$\neg B \neg B$	$A \wedge B A \wedge B$	$\neg B \wedge C \neg B \wedge C$	$(A \wedge B) \vee (\neg B \wedge C)$ $(A \wedge B) \vee (\neg B \wedge C)$

Two, possibly compound, statements A and B are considered to be **equivalent** if they have the same truth value for all inputs. This is denoted $A \leftrightarrow B$. **Figure 2.1.6** and **Figure 2.1.7** provide truth tables that show two pairs of equivalent statements known

as DeMorgan's Laws.

A	B	$A \wedge B$	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A \vee \neg B$	$\neg(A \wedge B) \leftrightarrow \neg A \vee \neg B$
0	0	0	1	1	1	1	1
0	1	0	1	1	0	1	1
1	0	0	1	0	1	1	1
1	1	1	0	0	0	0	1

Figure 2.1.6 DeMorgan's Law (and)

A	B	$A \vee B$	$\neg(A \vee B)$	$\neg A$	$\neg B$	$\neg A \wedge \neg B$	$\neg(A \vee B) \leftrightarrow \neg A \wedge \neg B$
0	0	0	1	1	1	1	1
0	1	1	0	1	0	0	1
1	0	1	0	0	1	0	1
1	1	1	0	0	0	0	1

Figure 2.1.7 DeMorgan's Law (or)

✓ Example 2.1.8: DeMorgan's Law.

Consider the statement: Guido likes to hike and cycle. Under what conditions is this statement false?

Solution

We want the negation. DeMorgan's Law tells us the negation is: Guido dislikes hiking or Guido dislikes cycling. This matches our expectation that a statement is false if any part of it is false.

Checkpoint 2.1.9

Are any of the statements in **Checkpoint 2.1.4** equivalent?

This page titled [2.1: Formal Logic](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

2.2: Circuits

Background

Digital circuits are built by combining large numbers of pieces, called gates. Gates take one or two electrical inputs which are either low (0) or high (1) current and output a single current. The gates act like negation, and, and or. **Figure 2.2.2** shows the symbols that are used on diagrams for these three gates.

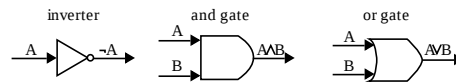


Figure 2.2.1 Symbols for Circuits

An example of combining gates to produce a more complicated circuit is shown in **Figure 2.2.2** for ‘if.’

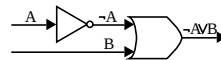


Figure 2.2.2 Circuit for If

Practice

Checkpoint 2.2.3

Draw the circuit diagram that accepts three inputs a,b,c and outputs $a \wedge (\neg b \vee c)$.

This page titled [2.2: Circuits](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

2.3: Predicate Logic

Background

In order to efficiently communicate logic statements we use predicates. These are simply functions with a codomain of ‘true’ and ‘false.’ For example $R(x) = x$ is red. Thus $R(stopsign) = true$ and $R(Seawolflogo) = false$. More complicated statements can be expressed as well. The following example converts the statement “ n is a positive, even number” into symbols using predicates.

Define the following statements.

$E(n)$ is the statement n is even.

$P(n)$ is the statement $n > 0$.

The statement: n is a positive, even number can now be written as $P(n) \wedge E(n)$.

Quantifiers

Terminology

For predicates we often ask the questions, “does any object satisfy the predicate (condition)” and “what is true of all objects that satisfy the predicate (condition).” The first we call **existential** and use the symbol $\exists$ for “there exists.” The second we call **universal** and use the symbol $\forall$ for “for all.” These two questions add **quantifiers** to logic. Another symbol used is $\ni$ for “such that.”

Consider the following predicates for examples of the notation.

$$E(n) = n \text{ is even} \quad (2.3.1)$$

$$P(n) = n \text{ is prime} \quad (2.3.2)$$

$$Q(n) = n \text{ is a multiple of } 4 \quad (2.3.3)$$

Using these predicates (symbols) we can express statements such as those in **Table 2.3.1**

Table 2.3.1. Quantified Statements

There is some number that is even and prime.	$\exists n \ni E(n) \wedge P(n)$
All multiples of four are even.	$\forall n Q(n) \rightarrow E(n)$

Checkpoint 2.3.2 Write the following statement using predicates and quantifiers. “All computer science majors must take Introduction to Discrete Mathematics and Data Structures and Algorithms.”

Negation

Negate a statement

One of the logical operations is negation. We can negate whole statements or parts of statements. A useful skill is to simplify the negation of a statement. This typically enables us to understand the meaning of the negation more easily and to optimize conditional statements.

✓ Example 2.3.3

This example demonstrates negating a statement involving conjunctions, but no quantifiers. Suppose the following claim is false.

“Guido owns a sail boat and has either a type rating for the Cessna Citation X or likes to cook.”

First we define predicates for each part of the statement.

$S(x) = x$ owns a sail boat.

$C(x) = x$ has a type rating for the Cessna Citation X.

$K(x)=x$ likes to cook.

Now we write the negation of this statement as

$\neg(S(\text{Guido}) \wedge (C(\text{Guido}) \vee K(\text{Guido}))) \leftrightarrow$ Using DeMorgan's Law we show this is the same as

$\neg S(\text{Guido}) \vee \neg(C(\text{Guido}) \vee K(\text{Guido})) \leftrightarrow$ Then we apply the other DeMorgan's Law to show it is the same as

$\neg S(\text{Guido}) \vee (\neg C(\text{Guido}) \wedge \neg K(\text{Guido})).$

Returning this to English gives us

"Either Guido does not own a sailboat, or he does not have the Cessna Citation X type rating and he does not like to cook."

Discover

In addition to DeMorgan's Laws (negation of **and** and **or**), the negations of the two quantifiers are needed. The negation of for all and there exists statements can be recognized from examples. Try the ones below.

Table 2.3.4 Friends

Name	Hair	Eye
Guido	Brown	Brown
Wolfgang	Red	Brown
Ludwig	Brown	Hazel
Franz	Brown	Brown
Maria	Gray	Brown

Checkpoint 2.3.5

How would you show that the statement "All of the five friends have brown hair" is false?

Checkpoint 2.3.6

How would you show that the statement "Some of the five friends have blue eyes" is false?

Checkpoint 2.3.7

What is the negation of the following statement. "All Apple fanboys own an iPad."

Checkpoint 2.3.8

What is the negation of the following statement. "Some chemist likes to cook."

Checkpoint 2.3.9

- For **Checkpoint 2.3.5** how many entries in the table did you have to check?
- For **Checkpoint 2.3.6** how many entries in the table did you have to check?
- What do your answers to these two questions tell you about the efficiency of the operators **and** and **or**?

Examples

✓ Example 2.3.10

We will write the statement: "Some flower is loved by all bees." First we must define all the terms.

$B(x)=x$ is a bee.

$F(x)=x$ is a flower.

$Y(x)=x$ is yellow.

$L(x,y)=x$ likes y .

Now we can write the statement in symbols. $\exists y F(y) \supset (\forall x B(x) \rightarrow L(x, y)).$

Next we will negate the statement. We will use knowledge of negating all/some and DeMorgan's laws.

$$\neg[\exists y F(y) \ni (\forall x B(x) \rightarrow L(x, y))] \leftrightarrow \forall y F(y) \rightarrow \neg(\forall x B(x) \rightarrow L(x, y)) \leftrightarrow \forall y F(y) \rightarrow (\exists x B(x) \ni \neg L(x, y)).$$

The negation is the statement: "Each flower is distasteful to some bee."

✓ Example 2.3.11

Next we will write the statement: "All red berries are loved by some bear."

$U(x)$ =x is a bear.

$B(x)$ =x is a berry.

$R(x)$ =x is red.

$L(x,y)$ =x likes y.

$$\forall y (B(y) \wedge R(y)) \rightarrow [\exists x U(x) \ni L(x, y)]$$

We can now negate the statement.

$$\neg(\forall y (B(y) \wedge R(y)) \rightarrow [\exists x U(x) \ni L(x, y)]) \leftrightarrow$$

$$\exists y (B(y) \wedge R(y)) \ni \neg[\exists x U(x) \ni L(x, y)] \leftrightarrow$$

$$\exists y (B(y) \wedge R(y)) \ni [\forall x U(x) \rightarrow \neg L(x, y)].$$

The negation is the statement: "Some red berry is not loved by any bear."

If you wonder why DeMorgan's law was not applied in the first step, look at the first example. When the $\exists$ (first clause) is negated, the exists statement becomes a for all statement and the negation is passed over the condition to the next clause. The negation changes exists/for all but does not change their condition.

Practice

Checkpoint 2.3.12

Negate the statement: "Either the person is a math major, or the person is a computer science major and has taken MATH A265."

Checkpoint 2.3.13

Negate: $\forall n C(n) \rightarrow (\exists m T(m) \ni [R(n, m) \vee S(n, m)])$. Meanings for the predicates are not required.

Checkpoint 2.3.14

Negate: $\forall x (A(x) \vee B(x)) \rightarrow (\exists y C(y) \ni [D(x, y) \vee E(x, y)])$. Meanings for the predicates are not required.

Checkpoint 2.3.15

We will negate a natural language statement in steps.

(a) Convert this statement to logic notation.

All moose love some mountain ash.

(b) Negate the statement using the logic notation.

(c) Convert your negated logic statement back to natural language.

Checkpoint 2.3.16

These demonstrate the full spectrum of what can happen and will require careful attention to grammar. Negate the following statements and re-write in English.

- $B(x)$: x is a bee.

- $F(x)$: x is a flower.
- $Y(x)$: x is yellow.
- $L(x,y)$: x likes y .

$$(a) \forall x B(x) \rightarrow (\forall y F(y) \rightarrow L(x, y)).$$

Answer

$$\exists x B(x) \ni (\exists y F(y) \ni \neg L(x, y)).$$

Some bee does not love some flower.

$$(b) \forall x B(x) \rightarrow (\exists y F(y) \ni L(x, y)).$$

Answer

$$\exists x B(x) \ni (\forall y F(y) \rightarrow \neg L(x, y)).$$

Some bee hates all flowers.

$$(c) \exists x B(x) \ni (\forall y F(y) \rightarrow L(x, y)).$$

Answer

$$\forall x B(x) \rightarrow (\exists y F(y) \ni \neg L(x, y)).$$

Every bee dislikes some flower.

$$(d) \exists x B(x) \ni (\forall y F(y) \rightarrow \neg L(x, y)).$$

Answer

$$\forall x B(x) \rightarrow (\exists y F(y) \ni L(x, y)).$$

Every bee loves some flower.

$$(e) \exists x B(x) \ni (\forall y [F(y) \wedge Y(y)] \rightarrow \neg L(x, y)).$$

Answer

$$\forall x B(x) \rightarrow (\exists y [F(y) \wedge Y(y)] \ni L(x, y)).$$

Every bee loves some yellow flower.

Checkpoint 2.3.17 The following two statements are not logically equivalent. Demonstrate why by finding a case that is true for one statement but false for the other statement.

- Some tree is loved by all moose.
- Every moose loves some tree.

► Hint

Consider whether each moose has to love the same tree.

Exercises

Use the following predicates below.

- $M(x)$: x is a moose.
- $G(x)$: x is a patch of grass.
- $N(x)$: x is new.
- $E(x,y)$: x eats y .

1. Write the following statements using predicate notation.

All moose eat some patch of new grass.

Some moose eats all patches of new grass.

Some moose eats no patch of new grass.

All moose do not eat some patch of new grass.

2. Negate the following statements using predicate notation.

All moose eat some patch of new grass.

Some moose eats all patches of new grass.

Some moose eats no patch of new grass.

All moose do not eat some patch of new grass.

3. Write the negations of the following statements in English.

All moose eat some patch of new grass.

Some moose eats all patches of new grass.

Some moose eats no patch of new grass.

All moose do not eat some patch of new grass.

4. What must you show to prove the following true?

All moose eat some patch of new grass.

Some moose eats all patches of new grass.

5. What must you show to prove the following false?

a. Some patch of new grass is eaten by no moose.

b. No moose does not eat some patch of grass.

This page titled [2.3: Predicate Logic](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

2.4: Mathematical Proof

Proof is an undefined term. It is also central to mathematics. Mathematicians, the group of people of whom you are currently part, think of proofs in two, related contexts. First proving a statement is about understanding that statement. When we can explain an idea based on already understood and accepted ideas, we understand the new statement. Second a proof is a means of communicating an understanding of a statement.

Note that the process of proving a statement is often inelegant and the result is often messy. At this point another trait of mathematicians comes into play: we love to produce better versions of things. We can take this messy result and clean it up to communicate it in a (more) elegant way.

First set proofs

Consider the statement: $A \cap B \subseteq A$. This statement is about being a subset. It begins with an intersection ($A \cap B$) and ends with another set (A). So we need to start with $x \in A \cap B$ and end up with $x \in A$. To bridge we have only the definition of intersection. It says $x \in A \cap B$ implies $x \in A$ and $x \in B$. The first part is all we need.

 **Theorem 2.4.1:** An intersection is a subset of either set.

For any two sets A and B , $A \cap B \subseteq A$.

Proof

Let $x \in A \cap B$. By definition of intersection $x \in A \cap B$ implies $x \in A$. Thus $x \in A \cap B$ implies $x \in A$. This is the definition of subset, so $A \cap B \subseteq A$.

Consider the statement: $A - B \subseteq \neg B$. This statement is also about being a subset. This begins with the set difference $A - B$ and ends with the set complement $\neg B$. So we need to start with $x \in A - B$ and end up with $x \in \neg B$. To bridge this we can use both the definition of set difference and the definition of set complement. The set difference definition tells us that $x \in A - B$ means $x \in A$ and $x \notin B$. The second part looks like set complement. But that requires that $x \in U$. Thus this will only work if $(A \subseteq U)$ (that is A and B have the same universal set).

 **Theorem 2.4.2:** A set difference is part of the complement.

For any two sets $A, B \subseteq U$, $A - B \subseteq \neg B$.

Proof

Let $x \in A - B$. By definition of set difference $x \notin B$. Because $(A \subseteq U)$, $a \in U$. Combining these two produces $x \in U - B = \neg B$. Thus $x \in A - B$ implies $x \in \neg B$. Hence $A - B \subseteq \neg B$.

Practice

Checkpoint 2.4.3

Prove $A \cap B \subseteq B$.

Checkpoint 2.4.4

Prove $A \cap B \subseteq A \cup B$.

This page titled [2.4: Mathematical Proof](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

3: Functions

[3.1: Functions](#)

[3.2: Boolean Algebra](#)

[3.3: Modulo Arithmetic](#)

[3.4: Combinatorics- Second Counts](#)

[3.5: Proof by Contradiction](#)

This page titled [3: Functions](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

3.1: Functions

Terminology

The idea of mapping one set of objects to another is basic to mathematics. **Mapping** and **correspondence** are undefined terms. We call the set mapped from the **domain** and the set mapped to the **codomain**. The **range** is the subset of the codomain to which a function is mapped.

Definition 3.1.1: Function.

A mapping is a **function** if and only if each input is mapped to exactly one output.

Definition 3.1.2: Onto.

A mapping is **onto** if and only if for each element of the codomain there exists some element of the domain mapped to it.

Onto functions are also referred to as **surjections**.

Definition 3.1.3: One-to-One.

A mapping is **one-to-one** if and only if $f(x) = f(a)$ implies $x = a$.

One-to-one functions are also referred to as **injections**.

Definition 3.1.4: Bijection.

A **bijection** is a function that is one-to-one (injective) and onto (surjective).

Definition 3.1.5: Inverse Mapping.

The **inverse mapping** for a mapping $f : D \rightarrow C$ is the mapping $f^{-1} : C \rightarrow D$ such that $f^{-1}(y) = x$ if and only if $f(x) = y$.

Note that the inverse mapping of a function is also a function if and only if the function is one-to-one.

Consider the examples in **Table 3.1.6** $D = \{-3, -2, -1, 0, 1, 2, 3\}$, $C = \{1, 1/2, 1/3, 1/4, 1/5, 1/6, 1/7\}$

Table 3.1.6 Functions

Name	Function	Onto	1-1
f	$f(1) = 1, f(2) = 1/2, f(3) = 1/3, f(4) = 1/4, f(5) = 1/5$	Yes	Yes
g	$g(1) = 1, g(2) = 1/2, g(3) = 1/3, g(4) = 1/2, g(5) = 1/3$	No	No
h	$h(1) = 1, h(2) = 1/2, h(3) = 1/3, h(4) = 1/6, h(5) = 1/5, h(6) = 1/7$	Yes	Yes

Because many functions will have numbers as their domains and/or codomains we use shorthand notation for many common sets of numbers. We denote the integers with Z . We denote the real numbers with R . If we want to refer to only positive numbers we use a $+$ superscript. For example Z^+ refers to the positive integers (also known as the counting numbers). Note as well that 0 is neither negative nor positive. If we want all the positive integers and zero we use $Z^+ \cup \{0\}$.

Practice

Checkpoint 3.1.7

Write a function from $D = \{-3, -2, -1, 0, 1, 2, 3\}$ to $C = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$

Checkpoint 3.1.8

Is your function onto? 1-1?

Checkpoint 3.1.9

Write a 1-1 function from D to C .

Checkpoint 3.1.10

Can you write a function from D onto C ?

Checkpoint 3.1.11

Write a function from Z onto $\{0, 1\}$.

This page titled [3.1: Functions](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

3.2: Boolean Algebra

3.2.1 Terminology

It is possible to process logic concepts using arithmetic and algebra concepts. To do so the following arithmetic definitions are needed. The first table defines addition the second defines multiplication and the third defines complement.

+	0	1
0	0	1
1	1	1

·	0	1
0	0	0
1	0	1

$$\begin{aligned}\overline{0} &= 1 \\ \overline{1} &= 0\end{aligned}$$

Figure 3.2.1 Boolean Functions

Checkpoint 3.2.2

Compare these three operations to the logical operations $\neg$, $\vee$, and $\wedge$.

Functions can be defined using boolean algebra as well. Consider the examples in **Table 3.2.3**.

Table 3.2.3 Boolean functions

x	y	$f_1(x, y) = x + y$	$f_2(x, y) = xy$	$f_3(x, y) = x + x$	$f_4(x, y) = \bar{x} + y$
0	0	0	0	0	1
0	1	1	0	0	1
1	0	1	0	1	0
1	1	1	1	1	1

Practice

Checkpoint 3.2.4

Evaluate the following boolean functions.

- $f_1(x, y) = 1 + x$
- $f_2(x, y) = \bar{x}\bar{y}$
- $f_3(x, y) = \bar{x} + \bar{y}$
- $f_4(x, y) = xy + y$

Checkpoint 3.2.5

A boolean function with two inputs, such as those above, has four input/output pairs. How many different boolean functions of two variables (inputs) exist? How many for a boolean function of three variables?

Checkpoint 3.2.6

Is the representation of a boolean function unique? This means for each of the functions counted above is there only one expression that produces that result?

Checkpoint 3.2.7

Find a function with the same input/output pattern as if x then y .

Checkpoint 3.2.8

Figure out the value for $f(x) = \overbrace{x + x + x + \dots}^{\text{\texttt{\textbackslash shortstackn}}}$ for all integers n .

Representation

Terminology

In complicated statements such as those that arise in complicated circuits, it is possible to optimize some statements by finding equivalent sets of operations. Boolean function equivalence can be used for this purpose.

First we will consider how to represent arbitrary boolean combinations as boolean functions. First, consider the expression in x , y , and z with the value 1 when $x = y = 1$ and $z = 0$. Because boolean multiplication is only 1 when all factors are one, this expression can be written as $xy\bar{z}$. Likewise an expression that is 1 when $x = y = z = 0$ is $\bar{x}\bar{y}\bar{z}$.

Next consider a function $f(x, y, z)$ defined by the following input/output table.

Table 3.2.9 Simplifying a boolean algebra

x	0	0	0	0	1	1	1	1
y	0	0	1	1	0	0	1	1
z	0	1	0	1	0	1	0	1
f(x,y,z)	1	0	0	0	0	0	1	0

Note that this function f is 1 exactly when one of the two expressions above ($xy\bar{z}$ and $\bar{x}\bar{y}\bar{z}$) is one. Because $u + v$ is 1 when either is one, $f(x, y, z) = (xy\bar{z} + \bar{x}\bar{y}\bar{z})$ is one way to represent this function.

While the mechanism presented above can be used to produce a boolean function or equivalent circuit for any desired combination of inputs and outputs, it does not necessarily produce the most efficient solution. The following example demonstrates how a boolean function can be simplified using the table of identities in **Table 3.2.10**

Table 3.2.10 Boolean Identities

$x + x$	=	x
$x \cdot x$	=	x
$x + 0$	=	x
$x + 1$	=	1
$x + \bar{x}$	=	1
$x\bar{x}$	=	0
$x \cdot 0$	=	0
$x \cdot 1$	=	x
$x + y$	=	$y + x$
xy	=	yx
$x + (y + z)$	=	$(x + y) + z$
$x(yz)$	=	$(xy)z$
$x(y + z)$	=	$xy + xz$
$x + yz$	=	$(x + y)(x + z)$
$x + xy$	=	x
$x(x + y)$	=	x
$\overline{x + y}$	=	$\bar{x} \cdot \bar{y}$
$\overline{xy}$	=	$\bar{x} + \bar{y}$

The following example demonstrates one way to simplify this boolean function.

$$\begin{aligned}
 f(x, y) &= x\bar{y} + xy + y\bar{x} \\
 &= x\bar{y} + yx + y\bar{x} \\
 &= x\bar{y} + y \cdot (x + \bar{x}) \\
 &= x\bar{y} + y \cdot 1 \\
 &= x\bar{y} + y \\
 &= y + x\bar{y} \\
 &= (y + x)(y + \bar{y}) \\
 &= (y + x) \cdot 1 \\
 &= y + x.
 \end{aligned}$$

Practice

Checkpoint 3.2.11

Find an expression that is 1 when $x = z = 1$ and $y = 0$. Use the technique above.

Checkpoint 3.2.12

Find a representation of the boolean function in **Table 3.2.13**

Table 3.2.13 Inputs and Outputs for Boolean Function

x	0	0	0	0	1	1	1	1
y	0	0	1	1	0	0	1	1
z	0	1	0	1	0	1	0	1
f(x,y,z)	0	1	0	1	1	0	0	0

Checkpoint 3.2.14

Remember that boolean functions can be represented as logic statements (and/or/negate) which can be represented as electrical circuits. Design the electrical circuit needed for a light with two switches using the following steps. Note the following supposes that the light is off when both switches are turned down (typical off).

1. Using 0 for down and 1 for up, determine which combinations of up and down results in the light being off (0) and on (1).
2. Write a boolean function for this using the technique above.
3. Convert this boolean function into a logic statement (using $\neg$, $\vee$, and $\wedge$).
4. Convert this boolean function into a circuit diagram.

Checkpoint 3.2.15

Reduce the boolean function $f(x, y, z) = xy\bar{z} + x\bar{y}z + \bar{x}y\bar{z}$.

This page titled [3.2: Boolean Algebra](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

3.3: Modulo Arithmetic

Terminology

All integers are said to be equivalent **mod** n to their remainder after dividing by n . For example $9 \equiv 4 \pmod{5}$ because $9/5$ has remainder 4. Similarly $13 \equiv 1 \pmod{2}$ because $13/2$ has remainder 1.

Example 3.3.1 shows a function evaluated (mod5).

✓ Example 3.3.1: Function mod 5.

n	0	1	2	3	4
n^2	0	1	4	4	1

The formal definition of modulo arithmetic requires two definitions.

✎ Definition 3.3.2: Divides.

a divides b , written $a|b$, if and only if there exists some $k \in \mathbb{Z}$ such that $ak = b$.

✎ Definition: Modulo.

a is equivalent to b modulo n , written $a \equiv b \pmod{n}$, if and only if $n|b - a$.

Note you may be familiar with the operation % in programming. That is an operation like $+$ or $\times$. Here we are using a different number system like integers or reals. Avoid conflating the two. For example % has an order of operation. The question of order does not make sense with arithmetic mod n as will be demonstrated.

Practice

To address the issue of order of operation under modulo arithmetic consider the function $f(n) = n^2 + 3n + 5 \pmod{7}$.

✓ Example 3.3.4: Calculate function modulo n , each step.

$$f(4) = 4^2 + 3(4) + 5 \pmod{7}.$$

First we calculate each term above $\pmod{7}$.

$$4^2 = 16 \equiv 2 \pmod{7}.$$

$$3(4) = 12 \equiv 5 \pmod{7}.$$

$$5 = 5 \equiv 5 \pmod{7}.$$

Next we add the terms $\pmod{7}$.

$$2 + 5 + 5 = 12 \equiv 5 \pmod{7}.$$

Thus

$$f(4) = 5 \pmod{7}.$$

✓ Example 3.3.5: Calculate function modulo n , last step.

$$f(4) = 4^2 + 3(4) + 5 \pmod{7}$$

$$= 33 \pmod{7}$$

$$\equiv 5 \pmod{7}.$$

Checkpoint 3.3.6

(a) Calculate $f(2)$ and $f(6)$ twice; once using **Example 3.3.4** and once using **Example 3.3.5**

(b) Does it matter whether we mod each term or only after the last step?

Checkpoint 3.3.7

Complete the table below calculating (mod 7).

n	0	1	2	3	4	5	6
$n^2 - 3$							

Checkpoint 3.3.8

Based on two numbers being equivalent modulo 5 if their remainders are the same after dividing by 5, list four (4) numbers that are equivalent to 3 (mod 5).

Checkpoint 3.3.9

Find all solutions to the following equation modulo 5. $n^2 - 1 = 0 \pmod{5}$.

Checkpoint 3.3.10

Find all solutions to $3n = 0 \pmod{5}$.

Checkpoint 3.3.11

Complete the table below calculating modulo 8.

n	0	1	2	3	4	5	6	7
$4n$								

Checkpoint 3.3.12

Find all solutions to $4n = 0 \pmod{8}$.

Checkpoint 3.3.13

Find all solutions to $5n = 0 \pmod{8}$.

Checkpoint 3.3.14

Using the graph paper in **Table 3.3.15**, graph the function $p(n) = n^2 - 4n + 3 \pmod{5}$.

Table 3.3.15 Graph Paper for function mod 5

4					
3					
2					
1					
0					
	0	1	2	3	4

Proofs

Theorem 3.3.16

If $a|b$ then $-a|b$.

Proof

By definition of divides $a|b$ implies there exists $k \in \mathbb{Z}$ such that $ak = b$.

Consider

$$\begin{aligned}(-a)(-k) &= \\ak &= b\end{aligned}$$

Thus $-a|b$ by definition of divides.

Lemma 3.3.17

If $a \equiv b \pmod{n}$ then $a - n \equiv b \pmod{n}$.

Proof

By definition of modulo equivalence $a \equiv b \pmod{n}$ implies that $n|(b - a)$. Thus, by definition of divides, there exists $k \in \mathbb{Z}$ such that $nk = b - a$.

Consider

$$\begin{aligned}b - (a - n) &= \\b - a + n &= nk + n \\&= n(k + 1).\end{aligned}$$

Thus $n|(b - (a - n))$ by definition of divides. This implies $a - n \equiv b \pmod{n}$.

This page titled [3.3: Modulo Arithmetic](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

3.4: Combinatorics- Second Counts

Terminology

Permutations, presented in Section [Subsection 1.5.1](#), count the number of ways to permute, or rearrange, objects. Note that when permuting objects, some can be ignored. For example [Table 3.4.1](#) are all permutations of two characters from the alphabet $\{a, b, c\}$.

Table 3.4.1 Permutations of $\{a, b, c\}$.

	ba	ca
ab		cb
ac	bc	

Sometimes the order of objects is unimportant. The number of outcomes can still be counted. These are called **combinations**. The combinations of two letters from the alphabet $\{a, b, c\}$ are ab, ac, bc.

Practice

Discover a Method

Checkpoint 3.4.2

How many permutations of two letters from the alphabet $\{a, b, c\}$ are there? How many combinations of two letters from the alphabet $\{a, b, c\}$ are there? How do you get the combinations count from the permutations count?

Checkpoint 3.4.3

How many permutations are there of three letters from the alphabet $\{a, b, c, d\}$? Select three of these letters. How many ways can you order these three letters? How does this help count the number of combinations of three letters from this alphabet?

Use the Method

Checkpoint 3.4.4

How many permutations of three letters from the English alphabet (26 characters) are there?

Checkpoint 3.4.5

If 10 people participate in a race and only the first three places are recorded, how many possible results are there?

Checkpoint 3.4.6

How many permutations of the letters of the word 'greet' are there?

Checkpoint 3.4.7

- Suppose 10 people apply for three, identical jobs. In how many orders can three of these 10 people be hired?
- How many ways are there to permute these three people?
- If the first problem counted the number of ways to order three (3) of ten (10) people and the second problem counted the number of ways any particular three can be ordered, how can these be combined to count the number of ways to select three (3) out of ten (10) people?

Checkpoint 3.4.8

How many ways are there to select two types of ice cream from a selection of 14?

Checkpoint 3.4.9

If a set has 15 elements, how many subsets of 4 elements exist?

This page titled [3.4: Combinatorics- Second Counts](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

3.5: Proof by Contradiction

The first proofs we worked started with the condition, applied definitions, and previous theorems, one by one until we arrived at the conclusion. These are called direct proof. Now we will look at the first alternate. In these proofs we begin by supposing our conclusion is wrong and then applying definitions and previous theorems one by one until we run into a problem.

First proof using contradiction

Definition 3.5.1: Mapping Composition.

For two mappings $g : A \rightarrow B$ and $f : B \rightarrow C$, the **composition** of those mappings, denoted $f \circ g$, is the mapping $h : A \rightarrow C$ defined by $h(x) = f(g(x))$

The following proof has two parts. The first is a direct proof, and the second is a proof by contradiction.

Theorem 3.5.2: The composition of two functions is a function.

If the mappings $g : X \rightarrow Y$ and $f : Y \rightarrow Z$ are functions, then the composition $h(x) = f(g(x))$ is a function.

Proof

First we show that for each element of the domain there exists an element in the codomain to which it is mapped. Let $x \in X$. Because g is a function there exists a $y \in Y$ such that $g(x) = y$. Because f is a function, there exists a $z \in Z$ such that $f(y) = z$. This means for all $x \in X$ there exists a $z \in Z$ such that $h(x) = z$.

Second we show that each element of the domain is mapped to a unique element of the codomain. Suppose for sake of contradiction that there exists $x \in X$ such that $h(x) = z_1$ and $h(x) = z_2$. This means that $f(g(x)) = z_1$ and $f(g(x)) = z_2$. Because g is a function there exists a unique $y \in Y$ such that $g(x) = y$. Thus $f(y) = z_1$ and $f(y) = z_2$. But this contradicts that f is a function which maps an element of the domain to a unique element of the codomain.

Thus the composition of two functions is a function.

This page titled [3.5: Proof by Contradiction](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

4: Relations

- [4.1: Example- Error Correcting Codes](#)
- [4.2: Mathematical Relations](#)
- [4.3: Equivalence Relations](#)
- [4.4: Partially Ordered Sets](#)
- [4.5: Combinatorics- Inclusion/Exclusion](#)

This page titled [4: Relations](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.1: Example- Error Correcting Codes

Illustrated

In order to transfer data, that data must first be encoded in some fashion. For example, letters are coded as numbers (Unicode). Every code is designed starting with the data and then assigning a code for each piece.

Additionally a code can address errors in transmission. Any time data is transmitted there is the chance that noise is introduced changing some of the values. To ensure that the correct data can be read despite the introduction of noise, codes are constructed that can either detect errors or correct some errors. To do this each piece of data is encoded with sufficient duplicate information so that limited noise cannot change one piece of data into another one. For these examples, the data will be considered to be binary strings. Errors will be counted as the numbers of bits that change.

Distance can be formally defined, but for these examples, we will use the specific distance defined by the distance between two binary strings as the number of digits that are different. For example $d(001, 011) = 1$ and $d(000, 110) = 2$. This is known as the **taxi cab** metric because it is the distance in blocks that must be traveled by a taxi. **Figure 4.1.1** is a diagram showing all 3 digit binary strings with lines connecting those that are distance 1.

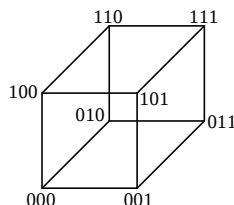


Figure 4.1.1 3 digit binary strings with distance one

Practice

The goal is to design a code such that we can be certain two binary strings are different.

Checkpoint 4.1.2

WebWork: <http://aholiab.uaa.alaska.edu/webwor...RPQ1VNRU5UKCk7>

Checkpoint 4.1.3

If a single bit of 00 or 11 were changed (0 to 1 or 1 to 0) could you be certain whether a 00 or 11 had been sent?

Checkpoint 4.1.4

WebWork: <http://aholiab.uaa.alaska.edu/webwor...RPQ1VNRU5UKCk7>

Checkpoint 4.1.5

WebWork: <http://aholiab.uaa.alaska.edu/webwor...VOVCgpOw%3D%3D>

Checkpoint 4.1.6

WebWork: <http://aholiab.uaa.alaska.edu/webwor...RPQ1VNRU5UKCk7>

Checkpoint 4.1.7

If a single bit of 000 or 111 were changed (0 to 1 or 1 to 0) could you be certain whether a 000 or 111 had been sent?

Checkpoint 4.1.8

For the sake of reading data, two strings are considered equivalent if we are sure that they represent the same initial data. If anything distance one away is considered equivalent, what are the other representations for 000 in the code above? What are the other representations of 111 in the code above?

Checkpoint 4.1.9

If two bits of 000 or 111 were changed (0 to 1 or 1 to 0) could you be certain whether a 000 or 111 had been sent?

Checkpoint 4.1.10

Build a code for the data 00, 01, 10, 11 that will correct one bit errors. What is the fewest number of bits that can be used for each piece of data providing single bit error correction?

This page titled [4.1: Example- Error Correcting Codes](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.2: Mathematical Relations

In order to organize data we need to recognize its underlying structure. **Relations** are one way of classifying structure.

Terminology

Definition 4.2.1: Relation.

A subset of a Cartesian product $R = S \times T$ defines a **relation** from S to T .

If $(s, t) \in R$ then s is related to t or sRt . Other notations are used to denote elements which are related for specific types of relations as will be seen later. Note a relation can be from a set to itself (e.g., $R = S \times S$).

For example consider the relation R on $Z^2 \times Z^2$ (Z^2 means order pairs of integers) defined by $(a, b)R(c, d)$ if and only if $ad = bc$. Thus $(1, 2)R(3, 6)$ because $1 \cdot 6 = 2 \cdot 3$. Also, $(5, 3)R(20, 12)$ because $5 \cdot 12 = 3 \cdot 20$.

Checkpoint 4.2.2

Find three elements related to $(2, 5)$ in this relation.

Consider the set $P(X)$ (power set) for some non-empty set X . Let the relation R be defined by A is related to B if and only if $A \subseteq B$. Consider $P(\{0, 1\}) = \{\emptyset, \{0\}, \{1\}, \{0, 1\}\}$. $\{0\}R\{0, 1\}$ because $\{0\} \subseteq \{0, 1\}$. Also $\emptyset R\{0\}$.

Checkpoint 4.2.3

Write $P(\{a, b, c\})$.

Checkpoint 4.2.4

Find all sets that are related to $\{a, b\}$ in the relation $(P(\{a, b, c\}), \subseteq)$.

Checkpoint 4.2.5

Find all sets that to which $\{a, b\}$ is related in the relation $(P(\{a, b, c\}), \subseteq)$.

Definition 4.2.6: Reflexive.

A relation R on a set X is **reflexive** if and only if $a \in X$ implies $(a, a) \in R$.

Consider the relation R on $Z^2 \times Z^2$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. For this relation to be reflexive (a, b) must be related to (a, b) . For this relation that requires that $ab = ba$. This is true by commutativity of real arithmetic. Thus this relation is reflexive.

Next consider the set $P(X)$ for some non-empty set X . Let the relation R be defined by A is related to B if and only if $A \subseteq B$. To show that this is reflexive (A, A) must be in R for all sets. In this case that means $A \subseteq A$ for all $A \in P(X)$. Note by a previous theorem $A \subseteq A$ for all sets, thus R is indeed reflexive.

Definition 4.2.7: Symmetric.

A relation R on a set X is **symmetric** if and only if $(a, b) \in R$ implies $(b, a) \in R$.

Consider the relation R on $Z^2 \times Z^2$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. For this relation to be symmetric if $(a, b)R(c, d)$ then $(c, d)R(a, b)$. For this relation, $(a, b)R(c, d)$ implies $ad = bc$. By commutativity of real multiplication $cb = da$ which means $(c, d)R(a, b)$. Thus this relation is symmetric.

Consider the set $P(X)$ for the set $X = \{a, b, c\}$. Let the relation R be defined by A is related to B if and only if $A \subseteq B$. To be symmetric if $A \subseteq B$ then $B \subseteq A$. However, note that $\emptyset \subseteq \{a, b, c\}$ so these are related. However $\{a, b, c\} \not\subseteq \emptyset$ so this relation is not symmetric.

 Definition 4.2.8: Anti-Symmetric.

A relation R on a set X is **anti-symmetric** if and only if $(a, b), (b, a) \in R$ implies $a = b$.

Consider the relation R on $Z^2 \times Z^2$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. If this is anti-symmetric then there will be no non-trivial symmetric pairs. Consider that $(1, 3)R(2, 6)$ and $(2, 6)R(1, 3)$ because $1 \cdot 6 = 2 \cdot 3$. However $(1, 3) \neq (2, 6)$. Thus this relation is not anti-symmetric.

Consider the set $P(X)$ for the set $X = \{a, b, c\}$. Let the relation R be defined by A is related to B if and only if $A \subseteq B$. Note that if $A \subseteq B$ and $B \subseteq A$ then by definition of set equality $A = B$. Thus this relation is anti-symmetric.

 Definition 4.2.9: Transitive.

A relation R on a set X is **transitive** if and only if $(a, b) \in R$ and $(b, c) \in R$ implies $(a, c) \in R$.

Consider the relation R on $Z^2 \times Z^2$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. For this relation to be transitive $(a, b)R(c, d)$ and $(c, d)R(e, f)$ then $(a, b)R(e, f)$. Note $(a, b)R(c, d)$ means $ad = bc$. Also $(c, d)R(e, f)$ means $cf = de$.

$$\begin{aligned} ad &= bc. \\ \frac{ad}{b} &= c. \\ cf &= de. \\ \frac{ad}{b}f &= de. \\ adf &= deb. \\ af &= eb \\ &= be. \end{aligned}$$

This last implies that $(a, b)R(e, f)$. Thus this relation is transitive.

Consider the set $P(X)$ for the set $X = \{a, b, c\}$. Let the relation R be defined by A is related to B if and only if $A \subseteq B$. To show that this is transitive requires showing that if $(A, B), (B, C) \in R$ then $(A, C) \in R$. For this relation this means that if $A \subseteq B$ and $B \subseteq C$ then $A \subseteq C$. This has been proven in a previous problem, thus the relation is transitive.

Practice

Checkpoint 4.2.10

Let the relation R on the set of integers be defined by aRb if and only if $a|b$. Using the definition of divides demonstrates that this relation is reflexive. You should be able to do it without specific numbers.

Checkpoint 4.2.11

Let the relation R on the set of integers be defined by aRb if and only if $a \equiv b \pmod{n}$. Using the definition demonstrate that R is reflexive using $n = 5$.

Checkpoint 4.2.12

Let the relation R on the set of real numbers be defined by aRb if and only if $a < b$. Prove that this relation is not reflexive.

Checkpoint 4.2.13

Let the relation R on the set of integers be defined by aRb if and only if $a|b$. Determine whether this relation is symmetric.

Checkpoint 4.2.14

Let the relation R on the set of integers be defined by aRb if and only if $a \equiv b \pmod{n}$. Determine whether this relation is symmetric.

Checkpoint 4.2.15

Let the relation R on the set of real numbers be defined by aRb if and only if $a \leq b$. Determine whether this relation is symmetric.

Checkpoint 4.2.16

Let the relation R on the set of integers be defined by aRb if and only if $a|b$. Determine whether this relation is anti-symmetric.

Checkpoint 4.2.17

Let the relation R on the set of integers be defined by aRb if and only if $a \equiv b \pmod{n}$. Determine whether this relation is anti-symmetric.

Checkpoint 4.2.18

Let the relation R on the set of real numbers be defined by aRb if and only if $a \leq b$. Determine whether this relation is anti-symmetric.

Checkpoint 4.2.19

Let the relation R on the set of integers be defined by aRb if and only if $a|b$. Determine whether this relation is transitive.

Checkpoint 4.2.20

Let the relation R on the set of integers be defined by aRb if and only if $a \equiv b \pmod{n}$. Determine whether this relation is transitive.

Checkpoint 4.2.21

Let the relation R on the set of real numbers be defined by aRb if and only if $a \leq b$. Determine whether this relation is transitive.

Closure

Definition 4.2.22: Closure.

The **closure** of a subset X of a cartesian product with respect to the symmetric or transitive property of a relation is the set Y such that $X \subseteq Y$ and Y is symmetric or transitive.

✓ Example 4.2.23

Find the symmetric closure of the relation R on the set X .

$$X = \{0, 1, 2, 3, 4\}. R = \{(0, 0), (0, 1), (1, 2), (2, 3), (3, 4)\}$$

Solution

$(0, 1) \in R$ requires adding $(1, 0)$

$(1, 2) \in R$ requires adding $(2, 1)$

$(2, 3) \in R$ requires adding $(3, 2)$

$(3, 4) \in R$ requires adding $(4, 3)$

The symmetric closure of R is $\{(0, 0), (0, 1), (1, 0), (1, 2), (2, 1), (2, 3), (3, 2), (3, 4), (4, 3)\}$

✓ Example 4.2.24

Find the transitive closure of the relation R on the set X .

$$X = \{0, 1, 2, 3, 4\}. R = \{(0, 0), (0, 1), (1, 2), (2, 3), (3, 4)\}$$

Solution

$(0, 1), (1, 2) \in R$ requires adding $(0, 2)$

$(0, 2), (2, 3) \in R$ requires adding $(0, 3)$

$(0, 3), (3, 4) \in R$ requires adding $(0, 4)$

$(1, 2), (2, 3) \in R$ requires adding $(1, 3)$

$(1, 3), (3, 4) \in R$ requires adding $(1, 4)$

$(2, 3), (3, 4) \in R$ requires adding $(2, 4)$

The transitive closure of R is $\{(0,0), (0,1), (0,2), (0,3), (0,4), (1,2), (1,3), (1,4), (2,3), (2,4), (3,4)\}$

Practice

Checkpoint 4.2.25

Construct the symmetric closure of the following relation. $X = \{a, b, c\}$ $R = \{(a, b), (b, c), (c, a)\}$.

Checkpoint 4.2.26

Construct the transitive closure of the following relation. $X = \{a, b, c\}$ $R = \{(a, b), (b, c), (c, a)\}$.

Checkpoint 4.2.27

Construct the closure under both symmetric and transitive properties of the following relation. $X = \{a, b, c\}$
 $R = \{(a, b), (b, c), (c, a)\}$.

Checkpoint 4.2.28

Why do we not define the anti-symmetric closure of a relation?

This page titled [4.2: Mathematical Relations](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.3: Equivalence Relations

With modulo arithmetic any number that is equivalent to another can be substituted in arithmetic without changing the result. In an error-correcting code any strings that are equivalent are interpreted as the same data. This section develops the properties that define equivalence in a general sense.

Definition 4.3.1: Equivalence Relation.

A relation is an **equivalence relation** if and only if the relation is reflexive, symmetric, and transitive.

Checkpoint 4.3.2

Determine whether the following relation is an equivalence relation. Consider the set $P(X)$ for the set $X = \{a, b, c\}$. Let the relation R be defined by A is related to B if and only if $A \subseteq B$. It may help to list all the elements of $P(X)$ first using the definition of power set.

Checkpoint 4.3.3

Determine whether the following relation is an equivalence relation. Let the relation R on the set of integers be defined by aRb if and only if $a \equiv b \pmod{n}$. Note the value of n is irrelevant here.

Checkpoint 4.3.4

Determine whether the following relation is an equivalence relation. Let the relation R on the set of integers be defined by aRb if and only if $a|b$.

Checkpoint 4.3.5

Determine whether the following relation is an equivalence relation. Let the relation R on the set of real numbers be defined by aRb if and only if $a \leq b$.

Equivalence Classes

Definition 4.3.6: Equivalence Class.

For an equivalence relation R , a set of elements, denoted $[y]$, is an **equivalence class** if and only if $[y] = \{x \mid xRy\}$.

The two questions we ask are

1. What are the equivalence classes?
2. What are the elements of an equivalence class.

Questions asking about elements of an equivalence class may be phrased any of the following ways.

- List elements from the equivalence class $[a]$
- Find elements of the equivalence class $[a]$
- Find the equivalence class $[a]$

Example 4.3.7: Find an Equivalence Class.

Consider the equivalence relation defined by aRb if and only if $a \cong b \pmod{5}$. Find the equivalence class $[0]$. List at least four elements.

Solution

$[0]$ is the set of all integers n such that $n \cong 0 \pmod{5}$. We calculated some of these before: $0, 5, 10, -5, \dots$. Thus $[0] = \{\dots, -5, 0, 5, 10, \dots\}$.

✓ Example 4.3.8: Find an Equivalence Class.

Consider the relation R on $Z^+ \times Z^+$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. Find the equivalence class $[(1, 2)]$. List at least four elements.

Solution

Based on previous calculations we know $(2, 4)R(1, 2)$, $(3, 6)R(1, 2)$, and $(-1, -2)R(1, 2)$. Thus $[(1, 2)] = \{\dots, (-1, -2), (1, 2), (2, 4), (3, 6), \dots\}$.

✓ Example 4.3.9: Find Distinct Equivalence Classes.

Consider the equivalence relation defined by aRb if and only if $a \cong b \pmod{5}$. List multiple, distinct equivalence classes.

Solution

Note $5 \nmid (1 - 0)$ so $[0]$ and $[1]$ are distinct equivalence classes. Also $5 \nmid (2 - 0)$ and $5 \nmid (2 - 1)$. Thus $[2]$ is a third, distinct equivalence class.

✓ Example 4.3.10: Find Distinct Equivalence Classes.

Consider the relation R on $Z^+ \times Z^+$ defined by $(a, b)R(c, d)$ if and only if $ad = bc$. List multiple, distinct equivalence classes.

Solution

Consider $(1, 2)$ and $(3, 5)$. Note $1 \cdot 5 \neq 2 \cdot 3$. Thus $[(1, 2)]$ and $[(3, 5)]$ are distinct equivalence classes. Also consider $(1, 7)$. Note $1 \cdot 7 \neq 2 \cdot 4$ and $3 \cdot 7 \neq 5 \cdot 4$. Thus $[(4, 7)]$ is a third, distinct equivalence class.

Checkpoint 4.3.11

Find the equivalence class of 0 and the class of 1 for the relation $a \equiv b \pmod{6}$.

Checkpoint 4.3.12

List every equivalence class for the relation $a \equiv b \pmod{6}$.

Checkpoint 4.3.13

Two five bit, binary code words are equivalent if they are distance two or less from one of the selected code words. List all the code words equivalent to 00000.

Checkpoint 4.3.14

For the code word problem above, what is the maximum number of equivalence classes possible?

Checkpoint 4.3.15

Consider the following equivalence relation. $X = \{mx + b \mid m, b \in R\}$, the set of all 2D lines. R is defined by $\ell_1(x) = m_1x + b_1 \equiv \ell_2(x) = m_2x + b_2$ if and only if $m_1 = m_2$.

Find the equivalence class of $y = 5x + 3$ and the class of $y = \frac{11}{7}x + \frac{13}{208}$.

Checkpoint 4.3.16

Consider the following equivalence relation. $X = \{mx + b \mid m, b \in R\}$, the set of all 2D lines. R is defined by $\ell_1(x) = m_1x + b_1 \equiv \ell_2(x) = m_2x + b_2$ if and only if $m_1 = m_2$.

List four distinct equivalence classes for this relation.

Characterizing Equivalence Classes

The following questions draw attention to properties of equivalence classes

Checkpoint 4.3.17

Suppose the equivalence class $[a] = \{a, c, e\}$. What is the equivalence class $[c]$?

Checkpoint 4.3.18

Suppose the equivalence class $[a]$ contains a, c, e, g and $[i]$ contains g . What else do we know is in $[i]$.

Checkpoint 4.3.19

If two equivalence classes share one element, how many elements do they share?

Checkpoint 4.3.20

Prove no even number is odd.

Checkpoint 4.3.21

Explain how even/odd defines an equivalence relation on the integers.

Checkpoint 4.3.22

Why does major not define an equivalence relation on the set of current students at UAA?

This page titled [4.3: Equivalence Relations](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.4: Partially Ordered Sets

Terminology

Definition 4.4.1: Poset.

A relation is an **partially ordered set** (poset) if and only if the relation is reflexive, anti-symmetric, and transitive.

Consider the following poset defined by listing the relations explicitly.

$R = \{(a, a), (a, d), (a, f), (a, g), (a, h), (b, b), (b, d), (b, e), (b, f), (b, g), (b, h), (c, c), (c, d), (c, f), (c, g), (c, h), (d, d), (d, f), (d, g), (d, h), (e, e), (e, g), (e, h), (f, f), (g, g), (h, h)\}$.

Another way to define the elements of the relation is a Hasse diagram. Figure 4.4.2 shows the Hasse diagram for R . The rules for constructing these diagrams are in List 4.4.3.

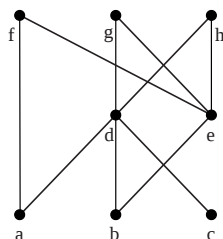


Figure 4.4.2 Hasse Diagram

List 4.4.3 Constructing a Hasse Diagram

- If $(a, b) \in R$ then a is below b on the diagram.
- If $(a, b) \in R$ then there is a line from a to b
 - unless $a = b$ (reflexive), or
 - unless $(a, c), (c, b) \in R$ (transitive set).

Note that constructing a Hasse diagram requires sorting the elements of the set. Each level of the diagram in some sense is “before” the levels above it. Because of this ordering in posets we write $a \preceq b$ to mean aRb .

Practice

Checkpoint 4.4.4

Determine whether the following relation is a poset. Let the relation R on the set of numbers $\{1, 2, 3, 6\}$ be defined by aRb if and only if $a|b$.

Checkpoint 4.4.5

Determine whether the following relation is a poset. Let the relation R on the set of numbers $\{1, 2, 3, 5, 6, 10, 15, 30\}$ be defined by aRb if and only if $a|b$.

Checkpoint 4.4.6

Determine whether the following relation is a poset. Let the relation R on the set of positive integers be defined by aRb if and only if $a|b$.

Checkpoint 4.4.7

Determine whether the following relation is a poset. Let the relation R on the set of real numbers be defined by aRb if and only if $a \leq b$.

Checkpoint 4.4.8

Determine whether the following relation is a poset. Consider the set $P(X)$ for the set $X = \{a, b, c\}$. Let the relation R be defined by A is related to B if and only if $A \subseteq B$.

Checkpoint 4.4.9

Draw the Hasse diagrams for the following posets.

1. $\subseteq$ on $P(\{0, 1\})$. Note it may help to list all the elements first.
2. $|$ (i.e., aRb iff $a|b$) on $\{1, 2, 3, 6\}$.
3. $|$ on $\{1, 2, 3, 5, 6, 10, 15, 30\}$.
4. $X = \{(0,0,0), (1,0,0), (0,1,0), (0,0,1), (0,1,1), (1,0,1), (1,1,0), (1,1,1)\}$ with the relation $(a_1, a_2, a_3) \preceq (b_1, b_2, b_3)$ iff $a_i \leq b_i$ for all i .

Poset Properties

Terminology

Definition 4.4.10: Comparable.

Two elements a, b of a poset are **comparable** if and only if either $a \preceq b$ or $b \preceq a$.

✓ Example 4.4.11: Comparable Elements.

For the poset represented by the Hasse diagram in **Figure 4.4.2** the elements d and g are *comparable*. The elements b and f are *incomparable*.

Definition 4.4.12: Total Ordering.

A partially ordered set is a **totally ordered** set if and only if all pairs of elements are comparable.

Definition 4.4.13: Least Upper Bound.

An element c is a **least upper bound** of two elements a and b , denoted $a \wedge b$, if and only if $a \preceq c$, $b \preceq c$ and if $a \preceq d$, $b \preceq d$ for any other element then $c \preceq d$.

✓ Example 4.4.14: LUB.

For the poset represented by the Hasse diagram in **Figure 4.4.2** $\text{lub}(a, b) = d$. Note h is another upper bound of a and b , but $d \preceq h$.

Definition 4.4.15: Greatest Lower Bound.

An element c is a **greatest lower bound** of two elements a and b , denoted $a \vee b$, if and only if $c \preceq a$, $c \preceq b$ and if $d \preceq a$, $d \preceq b$ for any other element then $d \preceq c$.

✓ Example 4.4.16: GLB.

For the poset represented by the Hasse diagram in **Figure 4.4.2** $\text{glb}(d, e) = b$. There are no other lower bounds for this pair.

Definition 4.4.17: Minimal Element.

An element a in a poset is a **minimal element** if and only if there does not exist an element b such that $b \preceq a$.

✓ Example 4.4.18: Minimal Element.

For the poset represented by the Hasse diagram in **Figure 4.4.2** the minimal elements are a, b , and c . a is a minimal element because there is no element x such that $x \preceq a$. Notice where the minimal elements are in the Hasse diagram.

 Definition 4.4.19: Maximal Element.

An element a in a poset is a **maximal element** if and only if there does not exist an element b such that $a \preceq b$.

✓ Example 4.4.20: Maximal Element.

For the poset represented by the Hasse diagram in **Figure 4.4.2** the maximal elements are f, g , and h . f is a maximal element because there is no element x such that $f \preceq x$.

Practice

Checkpoint 4.4.21

Find two comparable elements in each of the relations in **Checkpoint \(\PageIndex{9}\)**

Checkpoint 4.4.22

Find two incomparable elements in each of the relations in **Checkpoint \(\PageIndex{9}\)**

Checkpoint 4.4.23

Find all minimal and maximal elements of the posets in Problem **Checkpoint \(\PageIndex{9}\)**

Checkpoint 4.4.24

Consider the poset given in **Figure \(\PageIndex{2}\)** Find the following upper and lower bounds.

- (a) Find $f \vee h$
- (b) Find $a \wedge e$
- (c) Find $a \wedge d$
- (d) Does $d \wedge e$ exist?
- (e) Does $g \vee h$ exist?

 Definition 4.4.25: Lattice.

A poset in which every pair of elements has both a least upper bound and a greatest lower bound is a **lattice**

Checkpoint \(\PageIndex{26}\)

Which of the posets in **Checkpoint \(\PageIndex{9}\)** are lattices?

Checkpoint 4.4.27

Why is the name “lattice” appropriate?

This page titled [4.4: Partially Ordered Sets](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

4.5: Combinatorics- Inclusion/Exclusion

Terminology

Sometimes to count possibilities it is easier to over count and then count how much the over count was and fix it. For example, consider the size of the union of two sets A and B as shown in **Figure 4.5.1** Note that $|A| + |B|$ over counts the size of the union by precisely $|A \cap B|$. Thus the correct count is $(|A| + |B|) - |A \cap B|$.

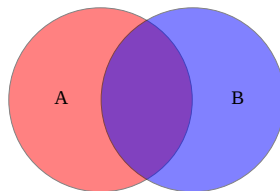


Figure 4.5.1 Count Elements of 2 Sets

This can be extended to three sets. See **Figure 4.5.2** This time there are three pairs of sets that overlap. Thus the count starts $(|A| + |B| + |C|) - (|A \cap B| + |A \cap C| + |B \cap C|)$. However this removes $|A \cap B \cap C|$ three times. Thus the correct count is $(|A| + |B| + |C|) - (|A \cap B| + |A \cap C| + |B \cap C|) + |A \cap B \cap C|$.

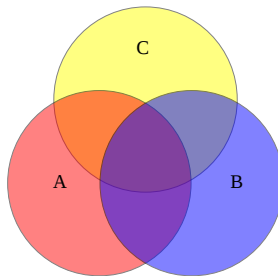


Figure 4.5.2 Count Elements of 3 Sets

Because this process first includes (adds) everything, next excludes (subtracts) some things, then includes (adds) other things, the technique is called ***inclusion/exclusion***.

✓ Example 4.5.3: Inclusion-Exclusion Count.

To use inclusion-exclusion to count we must know the size of each of the sets and each of the intersections. Suppose we know the following.

- 210 students in Calculus I
- 36 students in Discrete Math
- 32 students in Programming I
- 1 student is in Calculus I and Discrete Math
- 8 students are in Calculus I and Programming I
- 6 students are in Discrete Math and Programming I
- 0 students are taking all three

The calculation for the total number of (distinct) students is as follows.

$$\begin{aligned}
 |A| + |B| + |C| &= 210 + 36 + 32 &&= 278. \\
 |A \cap B| + |A \cap C| + |B \cap C| &= 1 + 8 + 6 &&= 15. \\
 |A \cap B \cap C| &= 0. \\
 |A \cup B \cup C| &= 278 - 15 + 0 &&= 263.
 \end{aligned}$$

 Definition 4.5.4: Derangement.

A permutation is a **derangement** if no element is in its original position.

For example “het” is a derangement of “the”, but “teh” is not a derangement.

Practice

Checkpoint 4.5.5

Write the formula for the number of elements in the union of four, distinct sets.

Checkpoint 4.5.6

Suppose you know that in the fall 2020 semester 37 distinct students took MATH A261, 25 distinct students took CSCE A311, and 11 distinct students took both. How many students total took either MATH A261 or CSCE A311 in the Fall 2020 semester?

Checkpoint 4.5.7

Count all derangements of “one”.

Checkpoint 4.5.8

Count all derangements of “four”.

Checkpoint 4.5.9

Count the number of positive multiples of 2,3,5 that are less than 100.

Checkpoint 4.5.10

If you know the following, what do you still need to know to determine the number of distinct students placing into MATH A054, WRTG A080, or having an SAT score below 1290?

1. Number of distinct students taking MATH A054.
2. Number of distinct students taking WRTG A080.
3. Number of distinct students with SAT score below 1290.

This page titled [4.5: Combinatorics- Inclusion/Exclusion](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

5: Graph Theory

5.1: Discovering Graphs

5.2: Properties of Graphs

5.3: Graph Isomorphism

5.4: Paths

5.5: Cycles

5.6: Trees

5.7: Linear Recurrence Relations

This page titled [5: Graph Theory](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.1: Discovering Graphs

Definitions are typically constructed after we work with new objects. The definition is constructed to match the properties we have observed and need. In this section we will practice this by developing a definition for a type of object known as a **graph**.

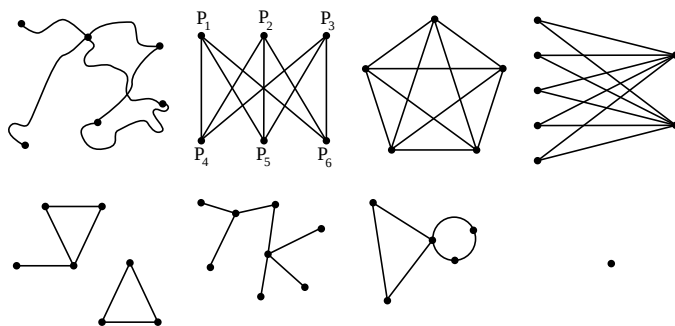


Figure 5.1.1: Each of these is a graph

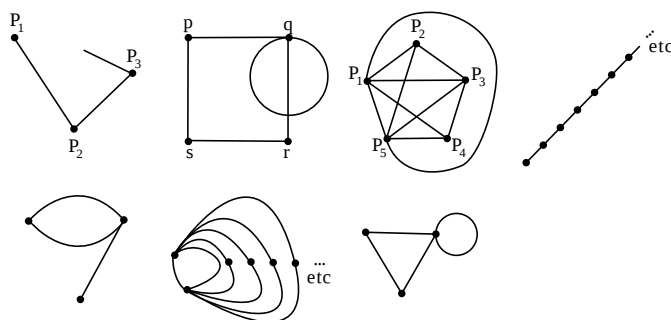
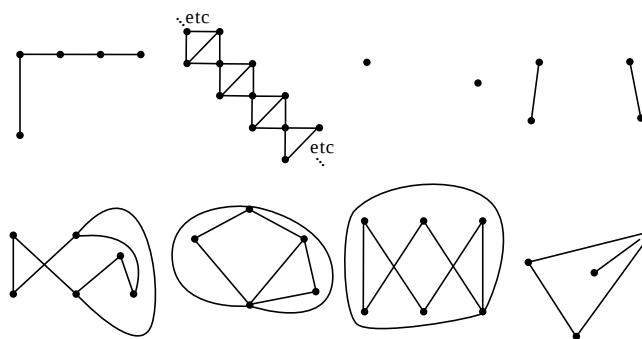


Figure 5.1.2: None of these is a graph

Checkpoint 5.1.3

Based on the examples in **Figure 5.1.1** and **Figure 5.1.2** determine which of the following are graphs.



✓ Example 5.1.4: Each of these is a graph.

Each of these is a graph.

1.

$$P = \{p_1, p_2, p_3\}$$

$$Q = \{q_1, q_2, q_3\}$$

$$q_1 = \{p_1, p_2\}$$

$$q_2 = \{p_1, p_3\}$$

$$q_3 = \{p_2, p_3\}$$

2.

3.

$$P = \{p_1, p_2, p_3, p_4\}$$

$$Q = \emptyset$$

$$P = \{p_1, p_2, p_3, p_4\}$$

$$Q = \{q_1, q_2\}$$

$$q_1 = \{p_1, p_3\}$$

$$q_2 = \{p_2, p_4\}$$
4.

$$P = \{p_1, p_2, p_3, p_4, p_5\}$$

$$L = \{\ell_1, \ell_2, \ell_3, \ell_4, \ell_5, \ell_6\}$$

$$\ell_1 = \{p_2, p_3\}$$

$$\ell_2 = \{p_1, p_4\}$$

$$\ell_3 = \{p_4, p_5\}$$

$$\ell_4 = \{p_1, p_5\}$$

$$\ell_5 = \{p_2, p_4\}$$

$$\ell_6 = \{p_2, p_5\}$$
5.

$$V = \{c, d, e, f\}$$

$$E = \{\{c, d\}, \{d, e\}, \{e, f\}, \{c, f\}\}$$
6.

$$V = \{v_1, v_2, v_3, v_4\}$$

$$E = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_2, v_3\}, \{v_2, v_4\}\}$$
7.

$$P = \{p\}$$

$$Q = \emptyset$$
8.

$$P = \{q, r, s, t\}$$

$$Q = \{\{q, r\}, \{r, t\}, \{q, t\}, \{r, s\}\}$$

✓ Example 5.1.5

None of these is a graph.

1.

$$P = \{p_1, p_2, p_3\}$$

$$Q = \{q_1, q_2\}$$

$$q_1 = \{p_1, p_2\}$$

$$q_2 = \{p_3, p_4\}$$
2.

$$P = \{p_1, p_2\}$$

$$Q = \{q_1, q_2\}$$

$$q_1 = \{p_1, p_2\}$$

$$q_2 = \{p_1, p_2\}$$
3.

$$P = \{p_1, p_2, p_3\}$$

$$Q = \{q_1, q_2, q_3\}$$

$$q_1 \& = (p_1, p_2)$$

$$q_2 = (p_1, p_3)$$

$$q_3 = (p_2, p_3)$$
4.

$$V = \{x, y, z\}$$

$$E = \{f, g, h\}$$

$$f = (x, y)$$

$$g = (y, x)$$

$$h = (y, z)$$

5. $P = \emptyset$
 $Q = \emptyset$
6. $P = \{q, r, s, t\}$
 $Q = \{\{q, r\}, \{r, t\}, \{s\}\}$
7. $V = \{p_1, p_2, \dots, p_n, \dots\}$
 $E = \{\{p_1, p_2\}, \{p_2, p_4\}, \{p_1, p_3\}, \{p_4, p_6\}\}$
8. $P = \{p_1, p_2, p_3\}$
 $Q = \{q_1, q_2, q_3\}$
 $q_1 = \{p_1, p_2\}$
 $q_2 = \{p_1, p_3\}$

Checkpoint 5.1.6

Based on the examples in Example 5.1.4 and Example 5.1.5 determine which of the following are graphs.

1. $P = \{p_1, p_2, p_3\}$
 $Q = \{q_1\}$
 $q_1 = \{p_1, p_3\}$
2. $P = \{p_1, p_2, p_3\}$
 $Q = \{q_1, q_2\}$
 $q_1 = \{p_1, p_2\}$
3. $V = \{v_1, v_2, \dots, v_n, \dots\}$
 $E = \{e_1, e_2, \dots, e_n, \dots\}$
 $e_1 = (v_1, v_2)$
 $\vdots$
 $e_n = (v_n, v_{n+1})$
 $\vdots$
4. $P = \{p, q, r\}$
 $Q = \{t, u, v\}$
 $t = \{p, q\}$
 $u = \{p, r\}$
 $v = \{q, r\}$
5. $P = \{p_1, p_2, p_3\}$
 $Q = \emptyset$
6. $V = \{u_1, u_2, u_3, u_4\}$
 $E = \{(u_1, u_2), (u_2, u_1), (u_3, u_4)\}$
7. $V = \{v_1, v_2, v_3\}$
 $E = \{\{v_1, v_2\}, \{v_1, v_3\}, \{v_3\}\}$
8. $P = \{p, q, r, s\}$
 $Q = \{\{p, q\}, \{p, r\}, \{p, s\}, \{q, r\}, \{q, s\}, \{r, s\}\}$

Checkpoint 5.1.7

In terms of the diagrams in Figure 5.1.1 list properties required and properties not allowed in graphs.

Checkpoint 5.1.8

In terms of the sets in Example 5.1.4 list properties required and properties not allowed in graphs.

Checkpoint 5.1.9

Draw the diagram form of the first graph in Example 5.1.4.

Checkpoint 5.1.10

Write the set form of the fourth graph (bottom left) in Figure 5.1.1.

This page titled [5.1: Discovering Graphs](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.2: Properties of Graphs

Definitions of Graph Properties

Definition 5.2.1: Adjacency.

Two vertices, v_i and v_j , in a graph G are adjacent if and only if $\{v_i, v_j\}$ is an edge in G .

✓ Example 5.2.2: Adjacent vertices.

Vertices A and B are **adjacent** in the graph in **Figure 5.2.11** because $\{A, B\}$ is an edge.

Definition 5.2.3: Incidence.

Two edges in a graph G are **incident** if and only if they share a vertex.

✓ Example 5.2.4: Incident Edges.

Edges $\{A, B\}$ and $\{B, G\}$ are **incident** in the graph in **Figure 5.2.11** because they share vertex B.

Definition 5.2.5: Vertex/Edge Incidence.

A vertex v and an edge $e = \{v_i, v_j\}$ in a graph G are **incident** if and only if $v \in e$.

✓ Example 5.2.6: Vertex Incident with Edge.

Vertex A is **incident** with edge $\{A, B\}$ in the graph in **Figure 5.2.11**, that is, A is in the edge.

Definition 5.2.7: Degree.

The **degree** of a vertex v is the number of edges incident with v .

✓ Example 5.2.8

Vertex A in the graph in **Figure 5.2.11** has degree 4, because $\{A, B\}$, $\{A, L\}$, $\{A, K\}$, and $\{A, F\}$ are edges incident with it.

The minimum degree of all vertices in a graph G is denoted $\delta(G)$ and the maximum degree of all vertices in a graph G is denoted $\Delta(G)$.

Definition 5.2.9: Regular.

A graph G is **regular** if and only if the degree of all vertices are the same.

✓ Example 5.2.10: Regular Graph.

Figure 5.2.11 shows a regular graph.

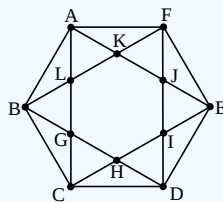


Figure 5.2.11 Regular Graph

 Definition 5.2.12: Complete Graph.

A graph G is a **complete** graph, denoted K_n , if and only if $\{v_i, v_j\} \in E$ for all $i \neq j$.

✓ Example 5.2.13: Complete Graph.

Figure 5.2.13 shows the complete graph, K_5 .

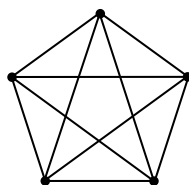


Figure 5.2.14 Complete Graph K_5

Note the **size** of a graph or subgraph is the number of vertices. Thus K_5 has size 5.

 Definition 5.2.15: Independent Set.

A subset A of vertices in a graph are **independent** if and only if no pair of vertices in A are adjacent.

✓ Example 5.2.16: Vertex Independence Set.

A, C, E is an independent set for the graph in Figure 5.2.11. Note no edge contains any two of these vertices.

 Definition 5.2.17: Bipartite Graph.

A graph G is **bipartite** if and only if the vertices can be partitioned into two sets such that no two vertices in the same partition are adjacent.

✓ Example 5.2.18: Understanding the bipartite definition.

This video shows how to demonstrate a graph is bipartite.



This video shows how to determine if a graph is bipartite.



This video shows how to demonstrate a graph is not bipartite.



 Definition 5.2.19: Complete Bipartite.

If each vertex in any partition of a bipartite graph is adjacent to all vertices in the other partition, the graph is called **complete bipartite** and is denoted $K_{n,m}$ where n, m are the sizes of the partitions.

✓ Example 5.2.20: Complete Bipartite Graph.

The third graph in Figure 5.2.44 is a complete bipartite graph.

Definition 5.2.21: Subgraph.

A graph $H = (V_H, E_H)$ is a **subgraph** of a graph $G = (V_G, E_G)$ if and only if $V_H \subseteq V_G$ and $E_H \subseteq E_G$.

Example 5.2.22: Subgraph.

The graph with vertex set $V_H = \{A, B, C, G, L\}$ and edge set $E = \{\{A, B\}, \{A, L\}, \{L, G\}, \{B, C\}, \{C, G\}\}$ is a subgraph of the graph in **Figure 5.2.11**

Definition 5.2.23: Induced Subgraph.

A graph $H = (V_H, E_H)$ is an **induced subgraph** of a graph $G = (V_G, E_G)$ if and only if $V_H \subseteq V_G$ and $E_H = \{(v_1, v_2) \in E_G : v_1, v_2 \in V_H\}$ (the set of all edges from G using only vertices in H).

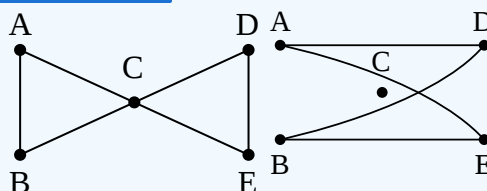
Example 5.2.24: Subgraph.

The graph with vertex set $V_H = \{A, B, C, G, L\}$ and edge set $E = \{\{A, B\}, \{A, L\}, \{B, L\}, \{B, G\}, \{L, G\}, \{B, C\}, \{C, G\}\}$ is the subgraph of the graph in **Figure 5.2.11** induced by V_H .

Definition 5.2.25: Graph Complement.

The **complement** of a graph $G = (V, E)$ is the graph $H = (V, E_2)$ such that v_1, v_2 are adjacent in H if and only if they are not adjacent in G .

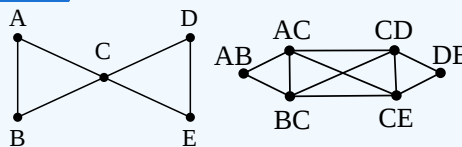
Example 5.2.26: A Graph and its Complement.



Definition 5.2.27: Graph Dual.

The **dual** of a graph $G = (V, E)$ is the graph $H = (E, E_2)$ such that for two vertices (edges of G) are adjacent if they were incident in G .

Example 5.2.28: A Graph and its Dual.



Practice

Checkpoint 5.2.29

List the minimum and maximum degree of every graph in **Figure 5.2.43**

Checkpoint 5.2.30

Determine which graphs in **Figure 5.2.43** are regular.

Complete graphs are also known as *cliques*. The complete graph on five vertices, K_5 , is shown in **Figure 5.2.14**. The size of the largest clique that is a subgraph of a graph G is called the *clique number*, denoted $\Omega(G)$.

Checkpoint 5.2.31

Find $\Omega(G)$ for every graph in **Figure 5.2.43**

Checkpoint 5.2.32

Prove that a complete graph is regular.

Checkpoint 5.2.33

Draw a graph with at least five vertices. Calculate the degree of each vertex. Add these degrees. Count the number of edges. Compare the sum of the degrees to the number of edges. Add an edge. Repeat the experiment. Conjecture a relationship.

Checkpoint 5.2.34

After the class confirms the result above prove that the number of vertices of odd degree is even.

The size of the maximum independent set in a graph G is denoted $\alpha(G)$.

Checkpoint 5.2.35

Find $\alpha(G)$ for every graph in **Figure 5.2.43**

Checkpoint 5.2.36

Re-write the definition of independent set exchanging vertices for edges. Note this is called a *matching*.

Checkpoint 5.2.37

Find the size of the maximum matching for each graph in **Figure 5.2.43**

Checkpoint 5.2.38

Determine which graphs in **Figure 5.2.43** are bipartite.

Checkpoint 5.2.39

Write a definition for tripartite graphs.

Checkpoint 5.2.40

Construct the graph complement of the bottom left graph in **Figure 5.2.44**

Checkpoint 5.2.41

Construct the graph complement of K_4 .

Checkpoint 5.2.42

Construct the dual graph of the bottom left graph in **Figure 5.1.1**.

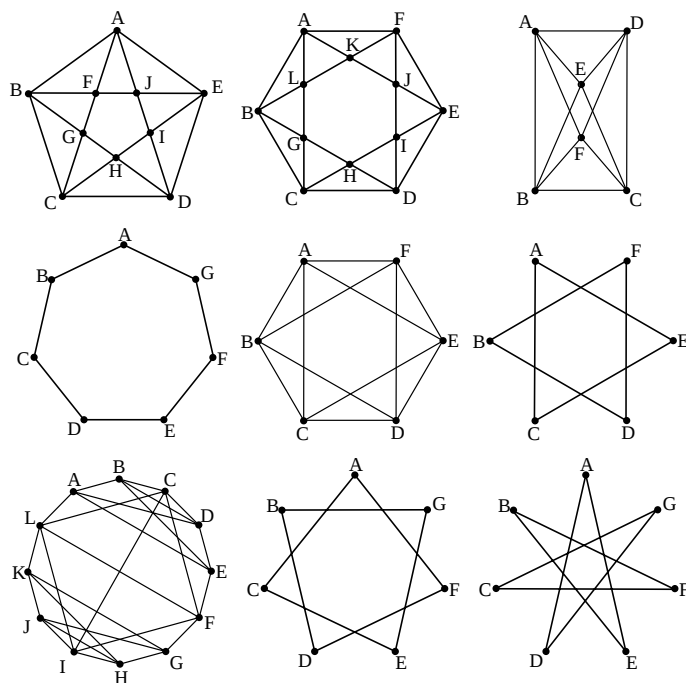


Figure 5.2.43 Graph Set 1

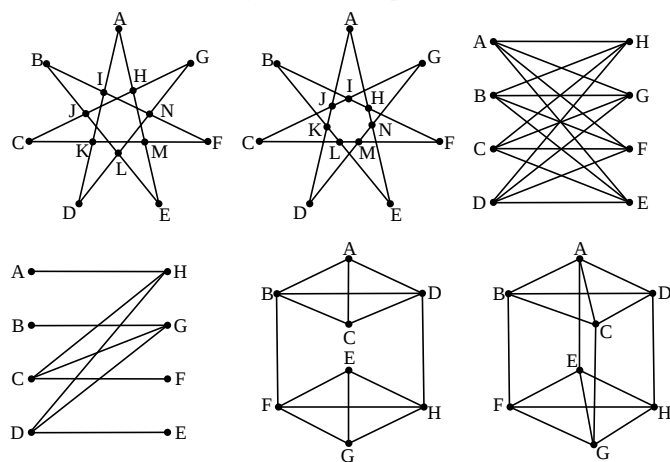


Figure 5.2.44 Graph Set 2

This page titled [5.2: Properties of Graphs](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.3: Graph Isomorphism

When calculating properties of the graphs in [Figure 5.2.43](#) and [Figure 5.2.44](#), you may have noted that some of the graphs shared many properties. It should also be apparent that a given graph can be drawn in many different ways given that the relative location of vertices and shape of edges is irrelevant. If two graphs are essentially the same, they are called **isomorphic**.

Definition 5.3.1: Graph Isomorphism.

Two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$ are *isomorphic* if and only if there exists a Bijection, called the *isomorphism*, $f : V_G \rightarrow V_H$ such that $\{v_1, v_2\} \in E_G$ if and only if $\{f(v_1), f(v_2)\} \in E_H$.

Example 5.3.2: Isomorphic Graphs.

The two graphs in **Figure 5.3.3** are isomorphic. The isomorphism is

$$A \mapsto I$$

$$B \mapsto J$$

$$C \mapsto L$$

$$D \mapsto K$$

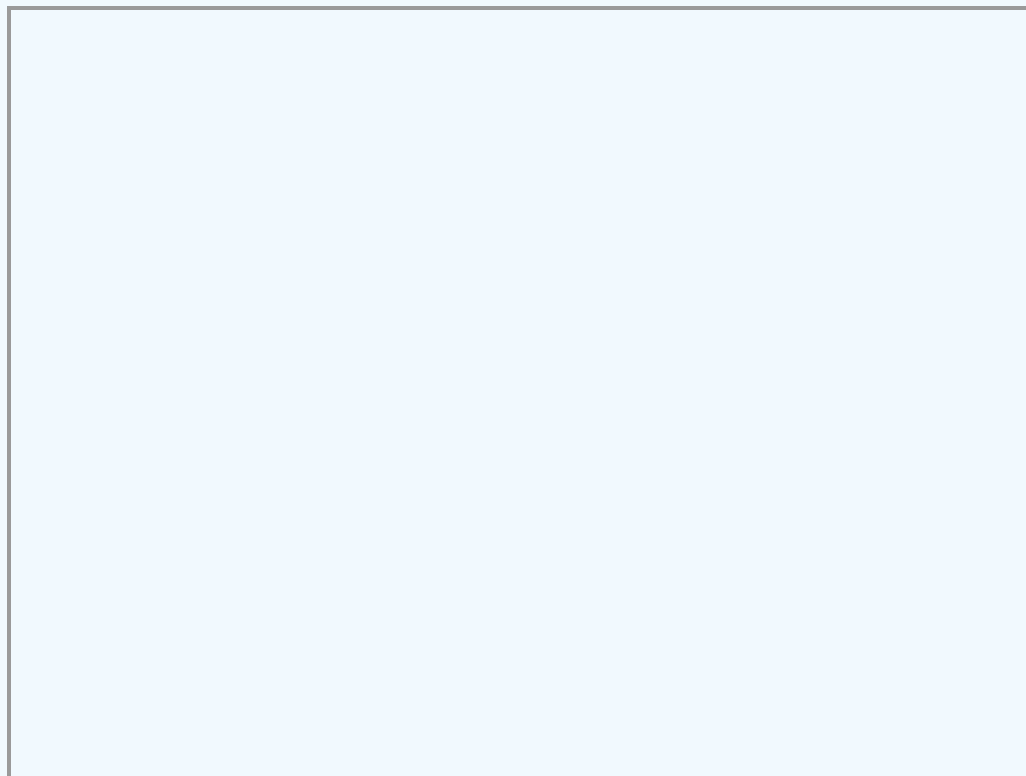
$$E \mapsto M$$

$$F \mapsto N$$

$$G \mapsto P$$

$$H \mapsto O$$

Drag the vertices of the graph on the left around until that graph looks like the graph on the right.



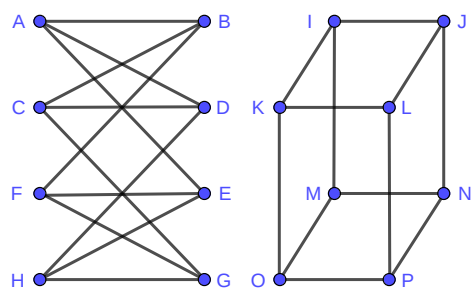


Figure 5.3.3 Two Isomorphic Graphs

Checkpoint 5.3.4

Determine which graphs in [Figure 5.2.43](#) and [Figure 5.2.44](#) are isomorphic. State the isomorphism (i.e., explicitly give the function).

This page titled [5.3: Graph Isomorphism](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.4: Paths

Paths

Definition 5.4.1: Walk.

A graph is a **walk** if and only if the vertices can be labeled $v_0, v_1, \dots, v_k$ such that v_i, v_{i+1} is an edge.

Definition 5.4.2: Trail.

A graph is a **trail** if and only if it is a walk such that no edge is used twice.

Definition 5.4.3: Path.

A graph is a **path** if and only if it is a walk such that no vertex is used twice.

A path on n vertices is denoted P_n . Please note that these terms (**walk**, **trail**, and **path**) vary widely including reversing the names, so always check the definition in the article, book, or other material you are reading.

Definition 5.4.4: Eulerian Trail.

A trail is **Eulerian** if and only if it uses every edge in the graph.

Definition 5.4.5: Hamiltonian Path.

A path is **Hamiltonian** if and only if it uses every vertex in that graph.

Practice

Checkpoint 5.4.6

Draw a path of length 5. Note this is denoted P_5 .

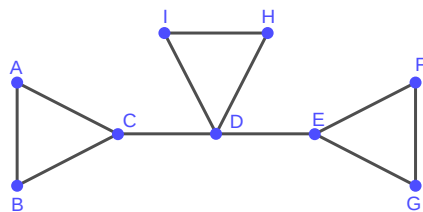


Figure 5.4.7 Graph Containing Walks, Trails, and Paths

Checkpoint 5.4.8

Find a walk that is not a trail in **Figure 5.4.7**

Checkpoint 5.4.9

Find a trail that is not a path in **Figure 5.4.7**

Checkpoint 5.4.10

Determine which graphs in **Figure 5.2.43** have Eulerian trails.

Checkpoint 5.4.11

Determine which graphs in **Figure 5.2.43** have Hamiltonian paths.

Connected

Definition 5.4.12: Vertex Connected.

A graph G is connected if and only if for every pair of vertices v, w there exists a path from v to w .

Definition 5.4.13: Vertex n -connected.

A graph G is **n -connected** if and only if removing any $n - 1$ vertices does not disconnect the graph.

Practice

Checkpoint 5.4.14

Determine which of the graphs in [Figure 5.1.1](#) are connected.

Checkpoint 5.4.15

Explain why every complete graph is connected.

Checkpoint 5.4.16

Explain why every complete bipartite graph is connected.

Checkpoint 5.4.17

Determine if every bipartite graph must be connected.

Checkpoint 5.4.18

For each of the graphs in [Figure 5.2.43](#) and [Figure 5.2.44](#) determine the maximum n for which the graph is n -connected.

Checkpoint 5.4.19

If a graph is n -connected what does this say about the minimum of the number of paths between any two vertices?

This page titled [5.4: Paths](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.5: Cycles

Terminology

Definition 5.5.1: Circuit.

A graph is a **circuit**, or **closed walk**, if and only if the vertices can be labeled $v_0, v_1, \dots, v_k$ such that $\{v_j, v_{j+1}\}$ is an edge for all j and $v_0 = v_k$.

Definition 5.5.2: Cycle.

A graph is a **cycle** if and only if it is a circuit such that no vertex is used twice.

A cycle on n vertices is denoted C_n . Like **walk** and **path** terminology for **cycles** varies widely.

Definition 5.5.3: Eulerian Circuit.

A circuit is **Eulerian** if and only if it uses every edge in the graph and no edge is used twice.

Definition 5.5.4: Hamiltonian Circuit.

A circuit is **Hamiltonian** if and only if it uses every vertex in that graph and no edge is used twice.

Practice

Checkpoint 5.5.5

Draw a cycle of length 4. Note this is denoted C_4 .

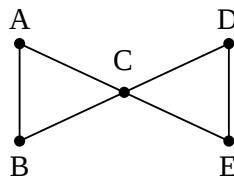


Figure 5.5.6 Graph Containing Circuits and Cycles

Checkpoint 5.5.5

Find a circuit that is not a cycle in **Figure 5.5.6**

Checkpoint 5.5.7

Determine which graphs in [Figure 5.2.43](#) have Eulerian circuits.

Checkpoint 5.5.9

Determine which graphs in [Figure 5.2.43](#) have Hamiltonian circuits.

This page titled [5.5: Cycles](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.6: Trees

Terminology

Definition 5.6.1: Tree

A graph is a **tree** if and only if it is connected and contains no cycles.

Definition 5.6.2: Leaf

A vertex in a tree is a **leaf** if and only if it has degree one.

Definition 5.6.3: Spanning Tree.

A graph G is a **spanning tree** of a graph H if and only if G is a subgraph of H that contains all the vertices of H and is a tree.

Practice

Checkpoint 5.6.4

Determine which graphs in Figure [Figure 5.1.1](#) are trees.

Checkpoint 5.6.4

Draw all non-isomorphic trees on 3 vertices.

Checkpoint 5.6.4

Prove that every tree with at least two vertices has at least one leaf.

Checkpoint 5.6.4

Draw a tree with 7 vertices. Determine for what n the tree is n -connected.

Checkpoint 5.6.4

Explain why for any tree removal of a leaf produces another tree.

Checkpoint 5.6.4

Find a spanning tree for every graph in [Figure 5.2.43](#).

This page titled [5.6: Trees](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

5.7: Linear Recurrence Relations

A couple interesting illustrations

Before defining terminology we will experiment with the following problems. Except where directed you should write code to perform the calculations so we concentrate on the results rather than the calculations.

Checkpoint 5.7.1

The following is called a continued radical.

1. Evaluate using technology $a_1 = \sqrt{2}$ to produce an approximation.
2. Take that result and calculate $a_2 = \sqrt{2 + a_1}$.
3. Take that result and calculate $a_3 = \sqrt{2 + a_2}$.
4. Repeat this seven (7) more times.
5. Is there a pattern to the numbers? What do you think will happen if you continue this process?

Checkpoint 5.7.2

The following is called a continued fraction.

1. Let $a_1 = 1$.
2. By hand calculate $a_2 = \frac{1}{1 + a_1}$.
3. By hand calculate $a_3 = \frac{1}{1 + a_2}$.
4. Repeat this seven (7) more times.
5. Is there a pattern to the numbers? What do you think will happen if you continue this process?
6. Ask the instructor to provide a number to compare.

Sometimes directly calculating the value of a function can be difficult. In the examples above each value of the function is based on the previous. The first example was $f(n) = \sqrt{2 + f(n-1)}$. The second example was $g(n) = \frac{1}{1 + g(n-1)}$. This is known as *recursive definition*.

Recurrence Relations

✓ Example 5.7.3: Fibonacci Sequence.

A famous example comes from Fibonacci's Liber Abaci.

The first month there was one pair of rabbits. The second month there was still one pair of rabbits. Starting the third month and continuing every month thereafter, the number of pairs of rabbits is the sum of the previous two months. Thus the third month there are $1 + 1 = 2$ pairs of rabbits. The fourth month there are $1 + 2 = 3$ pairs of rabbits. This produces the following sequence.

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

The Fibonacci sequence in standard notation is

$$a_1 = 1, a_2 = 1, a_n = a_{n-1} + a_{n-2}.$$

✓ Example 5.7.4: Counting: Restricted Strings.

It is possible to count using recursive functions as well. Consider the number of words from the alphabet $\{a, b, c, d, e\}$ with no two, consecutive a's. For words of length one there are no restrictions, so there are 5. For words of length two all combinations except 'aa' are acceptable. Thus there are $25 - 1 = 24$ words of length two. For words of length three all combinations except 'aaa', 'aaX', and 'Xaa' are allowable. This removes $1 + 4 + 4 = 9$ words leaving $5^3 - 9 = 116$ acceptable words.

For longer words of length n , note that removing the last letter produces an acceptable word of length $n - 1$. Appending b,c,d,e to the end of these length $n - 1$ words produces another acceptable word. This constructs $4a_{n-1}$ acceptable words of length n .

The only words to which an 'a' can be appended are those that ended in b,c,d, or e. These could have been appended to any word of length $n - 2$. Thus appending 'a' constructs an additional $4a_{n-2}$.

Thus the total number of acceptable words of length n is

$$a_n = 4a_{n-1} + 4a_{n-2}.$$

While it is possible to produce a function that provides the n th term, this is generally not easy. For linear recurrence relations the technique demonstrated here will always work.

Find a closed form for the recurrence relation

$$a_1 = 5, a_2 = 24, a_{n+2} = 4a_{n+1} + 4a_n.$$

$$a_{n+2} = 4a_{n+1} + 4a_n.$$

$$a^{n+2} = 4a^{n+1} + 4a^n. \text{ Switch to polynomial.}$$

$$a^2 = 4a + 4. \text{ Factor out shared powers.}$$

$$a^2 - 4a - 4 = 0.$$

Find the roots of polynomial.

$$a = \frac{-(-4) \pm \sqrt{(-4)^2 - 4(1)(-4)}}{2(1)}$$

$$= \frac{4 \pm \sqrt{16 + 16}}{2}$$

$$= \frac{4 \pm 2\sqrt{2}}{2}$$

$$= 2 \pm \sqrt{2}.$$

$$a_n = C_1(2 + \sqrt{2})^n + C_2(2 - \sqrt{2})^n. \text{ Setup closed form.}$$

Use known values to find coefficients.

$$a_1 = C_1(2 + \sqrt{2}) + C_2(2 - \sqrt{2}).$$

$$5 = C_1(2 + \sqrt{2}) + C_2(2 - \sqrt{2}).$$

$$a_2 = C_1(2 + \sqrt{2})^2 + C_2(2 - \sqrt{2})^2.$$

$$24 = C_1(6 + 4\sqrt{2}) + C_2(6 - 4\sqrt{2}).$$

$$C_1 = -1 + \frac{9}{4}\sqrt{2}.$$

$$C_2 = -1 - \frac{9}{4}\sqrt{2}.$$

$$a_n = \left(-1 + \frac{9}{4}\sqrt{2}\right)(2 + \sqrt{2})^n + \left(-1 - \frac{9}{4}\sqrt{2}\right)(2 - \sqrt{2})^n.$$

Practice

Checkpoint 5.7.5

Guido is superstitious and will not take food from two, adjacent bins at a buffet. If the buffet is a line having n bins each containing distinct food, how many different combinations of food can he choose? Note he can eat as many items as he wants.

Checkpoint 5.7.6

Guido walks up stairs taking one or two steps at a time. Find a recursive formula for the number of ways he could end up at step n . Note he starts at step 0 (not on the stairs).

This page titled [5.7: Linear Recurrence Relations](#) is shared under a [GNU Free Documentation License 1.3](#) license and was authored, remixed, and/or curated by [Mark A. Fitch](#) via [source content](#) that was edited to the style and standards of the LibreTexts platform.

CHAPTER OVERVIEW

6: Appendix

[6.1: A- University Database Example](#)

[6.2: B- Glossary](#)

[6: Appendix](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

6.1: A- University Database Example

Table A.0.1. Person

ID	8 digit integer, unique key
First Name	string
Last Name	string
Middle Name	string
Birth Year	integer
Birth Month	integer
Birth Day	integer
Ethnicity	string

Table A.0.2. Employee

ID	8 digit integer, unique key
Department	string
Position	string
Fund Source	string
Wage Type	string

Table A.0.3. Student

Enrolled	date
Admission Type	string
Admission Status	string
Class Standing	string
Major 1	string
Major 2	string
Catalog Year	4 digit integer

Table A.0.4. Course

CRN	digit integer
Year	4 digit integer
Semester	integer
Prefix	string length 4
Campus	string length 1
Course	string length 4
Section	string length 3

Table A.0.5. Transcript

ID	8 digit integer, unique key
CRN	digit integer
Year	4 digit integer
Semester	integer
Grade	string length 1
Last Day	date

6.1: A- University Database Example is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

6.2: B- Glossary

Symbol	Description	Location
Z	The set of the integers	Paragraph Because many functions will have numbers as their domains and/or codomains we use shorthand notation for many common sets of numbers. We denote the integers with Z. We denote the real numbers with R. If we want to refer to only positive numbers we use a + superscript. For example Z^+ refers to the positive integers (also known as the counting numbers). Note as well that 0 is neither negative nor positive. If we want all the positive integers and zero we use $Z^+ \cup \{0\}$. in-context
R	The set of the real numbers	Paragraph Because many functions will have numbers as their domains and/or codomains we use shorthand notation for many common sets of numbers. We denote the integers with Z. We denote the real numbers with R. If we want to refer to only positive numbers we use a + superscript. For example Z^+ refers to the positive integers (also known as the counting numbers). Note as well that 0 is neither negative nor positive. If we want all the positive integers and zero we use $Z^+ \cup \{0\}$. in-context

Z^+	The set of the positive integers	Paragraph Because many functions will have numbers as their domains and/or codomains we use shorthand notation for many common sets of numbers. We denote the integers with Z. We denote the real numbers with R. If we want to refer to only positive numbers we use a $+$ superscript. For example Z^+ refers to the positive integers (also known as the counting numbers). Note as well that 0 is neither negative nor positive. If we want all the positive integers and zero we use $Z^+ \cup \{0\}$. in-context
-------	----------------------------------	---

6.2: B- Glossary is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

Index

A

acceptable words
5.7: Linear Recurrence Relations

addition
3.2: Boolean Algebra

adjacency
5.2: Properties of Graphs

advanced features
InfoPage

affordable learning
InfoPage

algebra
3.2: Boolean Algebra

algorithm
1.3: Set Operations

alphabet
1.6: Sorting- Part 0

alphabetical
1.6: Sorting- Part 0

anti_symmetric
4.4: Partially Ordered Sets

arithmetic
3.2: Boolean Algebra

arrangements
1.5: Combinatorics- First Counts
3.4: Combinatorics- Second Counts

axioms
1.1: Terminology

B

bijection
3.1: Functions

binary strings
4.1: Example- Error Correcting Codes

bipartite
5.2: Properties of Graphs

bipartite graph
5.4: Paths

boolean functions
3.2: Boolean Algebra

booleanalgebra
2.2: Circuits
3: Functions

C

Cartesian product
1.3: Set Operations
4.2: Mathematical Relations

characters
1.5: Combinatorics- First Counts

checkpoints
1.5: Combinatorics- First Counts
1.6: Sorting- Part 0
5.1: Discovering Graphs

circuit
5.5: Cycles

circuitdiagram
2.2: Circuits

circuits
3.2: Boolean Algebra

coding theory
4.1: Example- Error Correcting Codes

codomain
3.1: Functions
3.5: Proof by Contradiction

codomains
6.2: B- Glossary

collection
1.2: Sets

combinations
3.4: Combinatorics- Second Counts
4.5: Combinatorics- Inclusion/Exclusion

combinatorics
1.5: Combinatorics- First Counts
3: Functions
4: Relations

combinatorylogic
2.2: Circuits

complement
2.4: Mathematical Proof
3.2: Boolean Algebra

complete
5.2: Properties of Graphs

composition
3.5: Proof by Contradiction

computer science majors
1.2: Sets
2.3: Predicate Logic

connected
5.4: Paths

construction
5.1: Discovering Graphs

continued fraction
5.7: Linear Recurrence Relations

continued radical
5.7: Linear Recurrence Relations

contradiction
3.5: Proof by Contradiction

corollaries
1.1: Terminology

counting
1.5: Combinatorics- First Counts
3.4: Combinatorics- Second Counts
4.5: Combinatorics- Inclusion/Exclusion
5.7: Linear Recurrence Relations

course
6.1: A- University Database Example

cycle
5.5: Cycles

cycles
5: Graph Theory

D

data
1.4: Sets in Relational Database

data structures
1.2: Sets
2.3: Predicate Logic

data transfer
4.1: Example- Error Correcting Codes

databases
1.4: Sets in Relational Database

date
6.1: A- University Database Example

defined terms

1.1: Terminology

definitions
1.1: Terminology
5.1: Discovering Graphs

degree
5.2: Properties of Graphs

DeMorgan's Law
2.1: Formal Logic
2.3: Predicate Logic

derangement
4.5: Combinatorics- Inclusion/Exclusion

difference
1.3: Set Operations

digital communication
4.1: Example- Error Correcting Codes

digital learning
InfoPage

digitalcircuits
2.2: Circuits

direct proof
3.5: Proof by Contradiction

discrete mathematics
2.3: Predicate Logic

DiscreteMath
4.5: Combinatorics- Inclusion/Exclusion

distance metric
4.1: Example- Error Correcting Codes

distinct
4.5: Combinatorics- Inclusion/Exclusion

domain
3.1: Functions
3.5: Proof by Contradiction

domains
6.2: B- Glossary

E

edge
5.2: Properties of Graphs

edges
5.4: Paths
5.5: Cycles

education
InfoPage

electricalinputs
2.2: Circuits

elegant
2.4: Mathematical Proof

elements
1.2: Sets

employee
6.1: A- University Database Example

empty set
1.2: Sets

encoding
4.1: Example- Error Correcting Codes

equivalence
4.3: Equivalence Relations

equivalence classes
4.3: Equivalence Relations

equivalence relation
4.3: Equivalence Relations

[equivalence relations](#)

[4: Relations](#)

[equivalents](#)

[2.1: Formal Logic](#)

[error correcting codes](#)

[4: Relations](#)

[error correction](#)

[4.1: Example- Error Correcting Codes](#)

[error detection](#)

[4.1: Example- Error Correcting Codes](#)

[Eulerian](#)

[5.4: Paths](#)

[5.5: Cycles](#)

[even numbers](#)

[2.3: Predicate Logic](#)

[examples](#)

[5.1: Discovering Graphs](#)

[exclusion](#)

[4.5: Combinatorics- Inclusion/Exclusion](#)

[exercise](#)

[3.4: Combinatorics- Second Counts](#)

[expressions](#)

[2.1: Formal Logic](#)

F

[Fibonacci sequence](#)

[5.7: Linear Recurrence Relations](#)

[function](#)

[3.1: Functions](#)

[functions](#)

[3: Functions](#)

[3.5: Proof by Contradiction](#)

G

[gates](#)

[2.2: Circuits](#)

[grade](#)

[6.1: A- University Database Example](#)

[graph](#)

[5.2: Properties of Graphs](#)

[5.4: Paths](#)

[5.5: Cycles](#)

[graphs](#)

[5: Graph Theory](#)

[5.1: Discovering Graphs](#)

[greatest_lower_bound](#)

[4.4: Partially Ordered Sets](#)

H

[Hamiltonian](#)

[5.4: Paths](#)

[5.5: Cycles](#)

I

[ID](#)

[6.1: A- University Database Example](#)

[illustrations](#)

[5.7: Linear Recurrence Relations](#)

[inclusion](#)

[4.5: Combinatorics- Inclusion/Exclusion](#)

[inclusion exclusion](#)

[4: Relations](#)

[injection](#)

[3.1: Functions](#)

[integers](#)

[1.2: Sets](#)

[3.1: Functions](#)

[4.2: Mathematical Relations](#)

[4.3: Equivalence Relations](#)

[6.2: B- Glossary](#)

[intersection](#)

[1.3: Set Operations](#)

[2.4: Mathematical Proof](#)

[isomorphic](#)

[5.3: Graph Isomorphism](#)

[isomorphism](#)

[5: Graph Theory](#)

J

[joins](#)

[1.4: Sets in Relational Database](#)

L

[least_upper_bound](#)

[4.4: Partially Ordered Sets](#)

[lemmas](#)

[1.1: Terminology](#)

[lexicographic](#)

[1.6: Sorting- Part 0](#)

[LibreTexts](#)

[InfoPage](#)

[linear recurrence relations](#)

[5: Graph Theory](#)

[5.7: Linear Recurrence Relations](#)

[logic](#)

[1.1: Terminology](#)

[2.1: Formal Logic](#)

[3.2: Boolean Algebra](#)

[logic statements](#)

[2.3: Predicate Logic](#)

[logicgates](#)

[2.2: Circuits](#)

M

[major](#)

[6.1: A- University Database Example](#)

[mapping](#)

[3.1: Functions](#)

[mappings](#)

[3.5: Proof by Contradiction](#)

[math](#)

[1.6: Sorting- Part 0](#)

[3.4: Combinatorics- Second Counts](#)

[mathematical notation](#)

[6.2: B- Glossary](#)

[mathematical relations](#)

[4: Relations](#)

[mathematics](#)

[1.1: Terminology](#)

[1.2: Sets](#)

[1.5: Combinatorics- First Counts](#)

[2.4: Mathematical Proof](#)

[3.1: Functions](#)

[mod n](#)

[3.3: Modulo Arithmetic](#)

[modular arithmetic](#)

[4.3: Equivalence Relations](#)

[moduloarithmetic](#)

[3: Functions](#)

[multiplication](#)

[3.2: Boolean Algebra](#)

N

[negation](#)

[2.1: Formal Logic](#)

[2.2: Circuits](#)

[2.3: Predicate Logic](#)

[noise](#)

[4.1: Example- Error Correcting Codes](#)

[notation](#)

[5.5: Cycles](#)

O

[objects](#)

[3.4: Combinatorics- Second Counts](#)

[OER](#)

[InfoPage](#)

[online platform](#)

[InfoPage](#)

[operations](#)

[1.4: Sets in Relational Database](#)

[2.1: Formal Logic](#)

[3.2: Boolean Algebra](#)

[ordering](#)

[1.6: Sorting- Part 0](#)

[outputs](#)

[2.2: Circuits](#)

P

[partially ordered sets](#)

[4: Relations](#)

[partially_ordered_set](#)

[4.4: Partially Ordered Sets](#)

[path](#)

[5.4: Paths](#)

[5.5: Cycles](#)

[paths](#)

[5: Graph Theory](#)

[permutation](#)

[1.5: Combinatorics- First Counts](#)

[permutations](#)

[1.6: Sorting- Part 0](#)

[3.4: Combinatorics- Second Counts](#)

[4.5: Combinatorics- Inclusion/Exclusion](#)

[person](#)

[6.1: A- University Database Example](#)

[poset](#)

[4.4: Partially Ordered Sets](#)

[positive integers](#)

[6.2: B- Glossary](#)

[possible orders](#)

[1.6: Sorting- Part 0](#)

[power set](#)

[1.3: Set Operations](#)

[4.3: Equivalence Relations](#)

[practice](#)

[1.2: Sets](#)

[1.5: Combinatorics- First Counts](#)

[1.6: Sorting- Part 0](#)

[2.1: Formal Logic](#)

[2.2: Circuits](#)

[5.1: Discovering Graphs](#)

[predicates](#)

[2.3: Predicate Logic](#)

prime numbers

[2.3: Predicate Logic](#)

probability

[3.4: Combinatorics- Second Counts](#)

problem solving

[1.1: Terminology](#)

proof

[2.4: Mathematical Proof](#)

[3.5: Proof by Contradiction](#)

proofbycontradiction

[3: Functions](#)

proofs

[1.1: Terminology](#)

[2.4: Mathematical Proof](#)

properties

[5: Graph Theory](#)

[5.1: Discovering Graphs](#)

[5.2: Properties of Graphs](#)

Q

quantifiers

[2.3: Predicate Logic](#)

query

[1.4: Sets in Relational Database](#)

R

range

[3.1: Functions](#)

real numbers

[4.2: Mathematical Relations](#)

[6.2: B- Glossary](#)

recurrence relations

[5.7: Linear Recurrence Relations](#)

recursive definition

[5.7: Linear Recurrence Relations](#)

reflexive

[4.2: Mathematical Relations](#)

[4.3: Equivalence Relations](#)

[4.4: Partially Ordered Sets](#)

regular

[5.2: Properties of Graphs](#)

relation

[4.2: Mathematical Relations](#)

[4.4: Partially Ordered Sets](#)

restricted strings

[5.7: Linear Recurrence Relations](#)

S

score

[1.4: Sets in Relational Database](#)

selections

[3.4: Combinatorics- Second Counts](#)

set difference

[2.4: Mathematical Proof](#)

set operations

[1.3: Set Operations](#)

set theory

[2.4: Mathematical Proof](#)

[5.1: Discovering Graphs](#)

[6.2: B- Glossary](#)

set_theory

[4.4: Partially Ordered Sets](#)

sets

[1.2: Sets](#)

[4.2: Mathematical Relations](#)

[4.5: Combinatorics- Inclusion/Exclusion](#)

SQL

[1.4: Sets in Relational Database](#)

statements

[2.1: Formal Logic](#)

strings

[1.5: Combinatorics- First Counts](#)

[1.6: Sorting- Part 0](#)

structure

[1.4: Sets in Relational Database](#)

[4.2: Mathematical Relations](#)

student

[6.1: A- University Database Example](#)

students

[1.4: Sets in Relational Database](#)

subgraph

[5.2: Properties of Graphs](#)

subset

[2.4: Mathematical Proof](#)

subsets

[1.2: Sets](#)

[1.3: Set Operations](#)

[3.4: Combinatorics- Second Counts](#)

surjection

[3.1: Functions](#)

symmetric

[4.2: Mathematical Relations](#)

[4.3: Equivalence Relations](#)

T

tables

[1.4: Sets in Relational Database](#)

taxi cab metric

[4.1: Example- Error Correcting Codes](#)

terminology

[1.3: Set Operations](#)

[2.1: Formal Logic](#)

[3.2: Boolean Algebra](#)

[5.5: Cycles](#)

textbooks

[InfoPage](#)

theorem

[2.4: Mathematical Proof](#)

[3.5: Proof by Contradiction](#)

theorems

[1.1: Terminology](#)

total_ordering

[4.4: Partially Ordered Sets](#)

trail

[5.4: Paths](#)

transcript

[6.1: A- University Database Example](#)

transitive

[4.2: Mathematical Relations](#)

[4.3: Equivalence Relations](#)

[4.4: Partially Ordered Sets](#)

Trees

[5: Graph Theory](#)

[5.6: Trees](#)

truth

[2.1: Formal Logic](#)

U

union

[1.3: Set Operations](#)

[4.5: Combinatorics- Inclusion/Exclusion](#)

unique key

[6.1: A- University Database Example](#)

uniqueness

[3.5: Proof by Contradiction](#)

V

Venn diagram

[1.3: Set Operations](#)

vertex

[5.2: Properties of Graphs](#)

[5.4: Paths](#)

vertices

[5.5: Cycles](#)

W

walk

[5.4: Paths](#)

words

[1.5: Combinatorics- First Counts](#)

Glossary

Sample Word 1 | Sample Definition 1

Detailed Licensing

Overview

Title: [Discrete Mathematics for Computer Science \(Fitch\)](#)

Webpages: 45

All licenses found:

- [GNU Free Documentation License](#): 86.7% (39 pages)
- [Undeclared](#): 13.3% (6 pages)

By Page

- [Discrete Mathematics for Computer Science \(Fitch\)](#) - [GNU Free Documentation License](#)
 - [Front Matter](#) - [GNU Free Documentation License](#)
 - [TitlePage](#) - [GNU Free Documentation License](#)
 - [InfoPage](#) - [GNU Free Documentation License](#)
 - [Table of Contents](#) - [Undeclared](#)
 - [Licensing](#) - [Undeclared](#)
 - [1: Basics](#) - [GNU Free Documentation License](#)
 - [1.1: Terminology](#) - [GNU Free Documentation License](#)
 - [1.2: Sets](#) - [GNU Free Documentation License](#)
 - [1.3: Set Operations](#) - [GNU Free Documentation License](#)
 - [1.4: Sets in Relational Database](#) - [GNU Free Documentation License](#)
 - [1.5: Combinatorics- First Counts](#) - [GNU Free Documentation License](#)
 - [1.6: Sorting- Part 0](#) - [GNU Free Documentation License](#)
 - [2: Logic](#) - [GNU Free Documentation License](#)
 - [2.1: Formal Logic](#) - [GNU Free Documentation License](#)
 - [2.2: Circuits](#) - [GNU Free Documentation License](#)
 - [2.3: Predicate Logic](#) - [GNU Free Documentation License](#)
 - [2.4: Mathematical Proof](#) - [GNU Free Documentation License](#)
 - [3: Functions](#) - [GNU Free Documentation License](#)
 - [3.1: Functions](#) - [GNU Free Documentation License](#)
 - [3.2: Boolean Algebra](#) - [GNU Free Documentation License](#)
 - [3.3: Modulo Arithmetic](#) - [GNU Free Documentation License](#)
 - [3.4: Combinatorics- Second Counts](#) - [GNU Free Documentation License](#)
 - [3.5: Proof by Contradiction](#) - [GNU Free Documentation License](#)
 - [4: Relations](#) - [GNU Free Documentation License](#)
 - [4.1: Example- Error Correcting Codes](#) - [GNU Free Documentation License](#)
 - [4.2: Mathematical Relations](#) - [GNU Free Documentation License](#)
 - [4.3: Equivalence Relations](#) - [GNU Free Documentation License](#)
 - [4.4: Partially Ordered Sets](#) - [GNU Free Documentation License](#)
 - [4.5: Combinatorics- Inclusion/Exclusion](#) - [GNU Free Documentation License](#)
 - [5: Graph Theory](#) - [GNU Free Documentation License](#)
 - [5.1: Discovering Graphs](#) - [GNU Free Documentation License](#)
 - [5.2: Properties of Graphs](#) - [GNU Free Documentation License](#)
 - [5.3: Graph Isomorphism](#) - [GNU Free Documentation License](#)
 - [5.4: Paths](#) - [GNU Free Documentation License](#)
 - [5.5: Cycles](#) - [GNU Free Documentation License](#)
 - [5.6: Trees](#) - [GNU Free Documentation License](#)
 - [5.7: Linear Recurrence Relations](#) - [GNU Free Documentation License](#)
 - [6: Appendix](#) - [Undeclared](#)
 - [6.1: A- University Database Example](#) - [Undeclared](#)
 - [6.2: B- Glossary](#) - [Undeclared](#)
 - [Back Matter](#) - [GNU Free Documentation License](#)
 - [Index](#) - [GNU Free Documentation License](#)
 - [Glossary](#) - [GNU Free Documentation License](#)
 - [Detailed Licensing](#) - [Undeclared](#)